Revisiting Exact Subgraph Counting [technical report]

Qiyan Li
The Chinese University of Hong Kong
Hong Kong, China
qyli@se.cuhk.edu.hk

Jeffrey Xu Yu
HKUST (GZ)
Guangzhou, China
jeffreyxuyu@hkust-gz.edu.cn

Hong Cheng
The Chinese University of Hong Kong
Hong Kong, China
hcheng@se.cuhk.edu.hk

***Abstract*—We study exact subgraph isomorphism (ISO) counting, which computes the occurrences of a pattern graph in a data graph as a global count or as local counts associated with individual nodes or edges. Under ISO semantics, different pattern nodes must map to different data nodes. Existing tree-decomposition-based tISO counting preserves injectivity within each subpattern of the decomposition and obtains the exact ISO count through a weighted combination of counts over the original pattern and multiple correction patterns, requiring extra processing of multiple patterns. Alternatively, enumerating complete matches enforces injectivity directly but loses the benefit of decomposition-based counting. We propose partial-match-based ISO counting, abbreviated as pISO, which evaluates only the original pattern without enumerating its complete matches. pISO selects a count set C of pairwise nonadjacent pattern nodes, constructs partial matches over the remaining nodes, and counts the injective mappings of C with an equation rather than enumerating them. We develop PTCount to choose between pISO and tISO by comparing the costs of generating partial matches and counting their injective extensions with those of evaluating multiple patterns. Its optimizer first applies lightweight structural rules and invokes cardinality-based cost estimation only for unresolved cases. Experiments show that PTCount outperforms state-of-the-art counting systems and selects the faster counting method quickly and accurately.**

I. Introduction

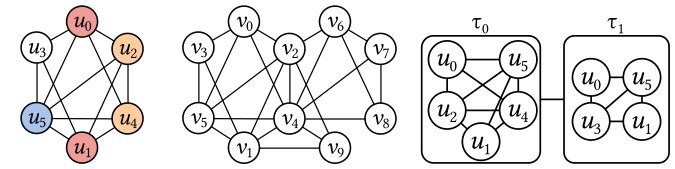
Subgraph counting computes the number of occurrences of a pattern graph p in a data graph G, where each occurrence maps the pattern nodes injectively to data nodes while preserving the pattern edges. It may return a global count or local counts associated with a node or edge orbit of p, referred to as node-orbit counting (VOC) and edge-orbit counting (EOC), respectively. These counts support graph clustering [1], network comparison and alignment [2], anomaly detection [3], detecting strong ties [4], and graph learning [5], [6].

A recent survey categorizes subgraph counting methods into enumeration-based, matrix-based, and decomposition-based approaches [7]. Enumeration-based methods explicitly visit complete matches; matrix-based methods derive target counts from equations over related patterns [8]–[13]; and decomposition-based methods count smaller subpatterns and combine their results [14]–[18]. Tree-decomposition-based methods are the most efficient among these approaches [19], [20]. A tree decomposition organizes overlapping subpatterns of a pattern as a tree. Each tree node stores a subset of pattern nodes, called a *bag*; its induced subgraph forms the bag's subpattern. These methods process the smaller bag subpatterns and combine their counts through nodes shared by adjacent bags, avoiding complete match enumeration. DISC computes an ISO count as a linear combination of homomorphism (HOM) counts [19]. SCOPE reduces the gap between the evaluated mappings and the desired injective mappings by introducing tISO counting [20]. In the tISO counting, bag subpatterns are evaluated by subgraph isomorphism, while aggregation is pushed across bags. Injectivity remains unchecked between nodes that never share a bag. Consequently, SCOPE obtains the exact ISO answer from a weighted combination of tISO counts for the original pattern and recursively generated correction patterns. Each term benefits from tree-decomposition-based aggregation, but every additional pattern requires its own decomposition processing. Table I reports the query counts and average evaluated pattern counts for all 5- and 6-node global, VOC, and EOC families. SCOPE improves on DISC by eliminating corrections for non-injective mappings detected within bags. For example, on 6-node VOC queries, SCOPE reduces the average number of terms from 9.654 to 8.600. However, the remaining correction terms must still be processed through their own tree decompositions. The need to process many additional patterns motivates a new method that evaluates only the original pattern.

TABLE I
AVERAGE PATTERN COUNT TERMS PER ISO QUERY.

Statistic	5-node patterns			6-node patterns		
	Global	VOC	EOC	Global	VOC	EOC
# queries	21	58	56	112	407	471
DISC (HOM)	3.905	4.948	4.929	7.205	9.654	10.146
SCOPE (tISO)	3.238	4.276	4.607	6.491	8.600	9.083

A straightforward way to ensure injectivity is to retain for every pattern node the data node to which it is mapped. However, this amounts to enumerating complete matches and loses the main benefit of decomposition-based counting. We address this limitation with partial-match-based ISO counting, abbreviated as pISO. It selects a *count set* C of pairwise nonadjacent pattern nodes and constructs partial matches for the remaining nodes through the tree decomposition. For each partial match, pISO builds candidate sets for the nodes in C, excludes data nodes already used by the partial match, and



(a) Pattern graph p (b) Data graph G (c) Tree decomposition T

vertex orbit

v_0	v_1	v_2	v_3	v_4	...
1	1	0	0	0	...

edge orbit

v_0 v_2	v_0 v_3	...	v_1 v_3	v_1 v_4	...
1	0	...	0	1	...

(d) Subgraph counts

Fig. 1. An example of subgraph counting.

applies a counting equation to obtain the injective mappings of C without enumerating them. Thus, pISO avoids both the correction patterns of tISO and the complete match enumeration of a conventional direct method.

The pattern structure and data graph determine the relative costs of pISO and tISO. tISO aggregates bag matches for the original and correction patterns, whereas pISO constructs partial matches of the original pattern and counts their injective extensions through candidate set operations. We develop PTCount, a tree-decomposition-based framework that chooses between these two methods for each ISO query. Its optimizer first applies inexpensive structural rules to eliminate clearly inferior plans and invokes cardinality-based cost estimation only when the structural comparison cannot decide.

Main Contributions. First, we introduce pISO, which constructs partial matches over the original pattern's tree decomposition and counts their injective extensions, avoiding correction patterns and complete match enumeration. Second, we optimize pISO with counting equations that eliminate or reduce candidate set intersections and compatible symmetry-breaking rules that prune partial match generation and candidate sets. Third, we develop PTCount with a two-step optimizer that selects between pISO and tISO using structural comparison and cardinality-based cost estimation. Finally, experiments show that PTCount outperforms state-of-the-art systems and that its optimizer selects counting methods quickly and accurately. The source code of PTCount is publicly available [21].

Organization. Section II introduces the basic concepts. Section III presents pISO counting. Section IV analyzes the time and space complexity of pISO. Section V presents the cost model and two-step counting method optimizer. Sections VI and VII discuss related work and experimental evaluation, and Section VIII concludes the paper.

II. Preliminaries

Graph model. We model a graph as a simple undirected graph $G = (V, E)$, where V is the node set and $E \subseteq \{\langle u, v\rangle \mid u, v \in V, u \neq v\}$ is the edge set. Unless otherwise stated, $\langle u, v\rangle$ denotes an undirected edge. There are no self-loops or multiple edges. We use $N_G(u) = \{v \mid \langle u, v\rangle \in E\}$ to denote the neighbors of u in G, and omit the subscript when the graph is clear. The numbers of nodes and edges are $n = |V|$ and $m = |E|$, respectively. For $S \subseteq V$, $G[S]$ denotes the subgraph induced by S, i.e., $G[S] = (S, \{\langle u, v\rangle \in E \mid u, v \in S\})$. A graph $G' = (V', E')$ is a subgraph of G if $V' \subseteq V$ and $E' \subseteq E$, and is an induced subgraph of G if $G' = G[V']$.

Homomorphisms and subgraph isomorphisms. Let $p = (V_p, E_p)$ be a connected pattern graph and let $G = (V, E)$ be a data graph. We use the terms *mapping* and *match* interchangeably throughout this paper. A mapping $f : V_p \mapsto V$ is a *homomorphism* from p to G if every pattern edge is preserved, i.e., $\langle f(u), f(u')\rangle \in E$ for every $\langle u, u'\rangle \in E_p$. We denote the set of all such mappings by $\mathsf{HOM}(p, G)$. A homomorphism f is a *subgraph isomorphism* if it is injective: $f(u) \neq f(u')$ for all distinct $u, u' \in V_p$. We denote the set of subgraph isomorphisms by $\mathsf{ISO}(p, G)$.

Each mapping f produces a graph $G_f = (V_f, E_f)$, where $V_f = \{f(u) \mid u \in V_p\}$ and $E_f = \{\langle f(u), f(u')\rangle \mid \langle u, u'\rangle \in E_p\}$. If $f \in \mathsf{ISO}(p, G)$, then G_f is its matched subgraph.

Automorphisms and orbits. For a graph $p = (V_p, E_p)$, an automorphism is a bijection $\gamma : V_p \mapsto V_p$ that preserves adjacency. The set of all automorphisms forms the automorphism group $\mathsf{Aut}(p)$. Two nodes $u, u' \in V_p$ are in the same node orbit iff there exists $\gamma \in \mathsf{Aut}(p)$ such that $\gamma(u) = u'$. Node orbits partition V_p, grouping nodes with the same topological role in p. We denote a node orbit by ϑ and select a representative $o \in \vartheta$; when the context is clear, we refer to the orbit by its representative o.

Automorphisms also act on edges. Two edges $\langle u_1, u_2\rangle$ and $\langle u_1', u_2'\rangle$ are in the same edge orbit iff there exists $\gamma \in \mathsf{Aut}(p)$ such that $\gamma(u_1) = u_1'$ and $\gamma(u_2) = u_2'$. We denote an edge orbit by ω and select a representative $\langle o, o'\rangle \in \omega$

Example 2.1: Figure 1 shows a running example. The assignment $f(u_i) = v_i$ for $0 \leq i \leq 5$ maps p to the left part of G. It preserves every pattern edge and is injective; hence f is a subgraph isomorphism and therefore also a homomorphism. In contrast, the assignment $g : (u_0, u_1, u_2, u_3, u_4, u_5) \mapsto (v_6, v_6, v_7, v_2, v_8, v_4)$ still preserves every pattern edge, but maps both u_0 and u_1 to v_6; thus g is a homomorphism but not a subgraph isomorphism. The pattern has four automorphisms, obtained by independently choosing whether to swap u_0 with u_1 and whether to swap u_2 with u_4. The node colors in the pattern illustrate these automorphism orbits: u_0 and u_1 are symmetric, and u_2 and u_4 are symmetric. □

Symmetry-breaking. Symmetry-breaking reduces duplicate mappings caused by pattern automorphisms [22], [23]. Given a fixed total order on data nodes, the rule $\theta = \{u_1 < u_i \mid 2 \leq i \leq k\}$ for an orbit $\vartheta = \{u_1, \ldots, u_k\}$ requires the data node matched to u_1 to be smaller than those matched to the other nodes in ϑ. These constraints select one representative among mappings that differ only by pattern automorphisms.

Mapping counts, rooted counts, and subgraph counts. Subgraph counting may count mappings or distinct matched subgraphs. Matched subgraph counts avoid counting the same data-side subgraph multiple times due to pattern automorphisms. For an unrooted pattern, $|\mathsf{SubG}(p, G)| = |\mathsf{ISO}(p, G)|/|\mathsf{Aut}(p)|$.

Let ϑ be a node orbit of p and let $o \in \vartheta$ be a representative node. For a data node $v \in V$, we define $\mathsf{ISO}^G_{p,o}(v) = \{f \in \mathsf{ISO}(p, G) \mid f(o) = v\}$. The node orbit count of v with respect to (p, ϑ) is the number of matched subgraphs in which a node in ϑ is mapped to v, denoted by $|\mathsf{SubG}^G_{p,o}(v)|$. Similarly, let ω be an edge orbit of p and let $\langle o, o' \rangle \in \omega$ be a representative edge. For a data edge $e \in E$, we define $\mathsf{ISO}^G_{p,\langle o,o' \rangle}(e) = \{f \in \mathsf{ISO}(p, G) \mid f(\langle o, o' \rangle) = e\}$. The edge orbit count of e with respect to (p, ω) is the number of matched subgraphs in which $\langle o, o' \rangle$ is mapped to e, denoted by $|\mathsf{SubG}^G_{p,\langle o,o' \rangle}(e)|$.

Following the standard orbit counting relation used in local subgraph counting [19], [20], SubG counts and ISO counts differ only by automorphism factors. Specifically,

$$|\mathsf{SubG}^G_{p,o}(v)| = |\mathsf{ISO}^G_{p,o}(v)| \cdot \frac{|\vartheta|}{|\mathsf{Aut}(p)|}. \tag{1}$$

For $\langle o, o' \rangle \in \omega$, the corresponding relation is

$$|\mathsf{SubG}^G_{p,\langle o,o' \rangle}(e)| = |\mathsf{ISO}^G_{p,\langle o,o' \rangle}(e)| \cdot \frac{|\omega|}{|\mathsf{Aut}(p)|}. \tag{2}$$

Problem statement. This paper studies exact subgraph counting. Given a data graph G, a pattern graph p, and an aggregation mode, the task is to compute the requested distinct matched subgraph counts exactly. Global counting returns $|\mathsf{SubG}(p, G)|$. Given a node orbit ϑ with representative o, node-orbit counting returns $|\mathsf{SubG}^G_{p,o}(v)|$ for every $v \in V$. Given an edge orbit ω with representative $\langle o, o' \rangle$, edge-orbit counting returns $|\mathsf{SubG}^G_{p,\langle o,o' \rangle}(e)|$ for every $e \in E$. We propose pISO counting, which counts injective extensions of partial matches of the original pattern. We also study selecting between pISO and tISO using pattern structure and cost estimation.

Example 2.2: Continuing Example 2.1, composing the mapping f with the four automorphisms of p yields four ISO mappings, all of which induce the same data-side subgraph in G. The symmetry-breaking rule set $\Theta = \{u_0 < u_1,\ u_2 < u_4\}$ requires a mapping f to satisfy $f(u_0) < f(u_1)$ and $f(u_2) < f(u_4)$. Among the four mappings, the mapping f, where $f(u_i) = v_i$, satisfies both rules, so it is the unique representative of these automorphic mappings. Thus, the left matched subgraph is counted once. The node-orbit count shown in the figure is rooted at the red node orbit of p. It is 1 for v_0 and v_1 because the unique matched subgraph maps some red pattern node to each of these data nodes. For v_0, two root-preserving subgraph isomorphisms map u_0 to v_0. They differ only by swapping the data nodes assigned to u_2 and u_4. Since the red-node orbit has size 2 and $|\mathsf{Aut}(p)| = 4$, Eq. (1) gives $|\mathsf{SubG}^G_{p,u_0}(v_0)| = |\mathsf{ISO}^G_{p,u_0}(v_0)| \cdot 2/4 = 1$. Similarly, consider the edge-orbit count that is rooted at the red–orange edge orbit. It is 1 for edges $\langle v_0, v_2 \rangle$ and $\langle v_1, v_4 \rangle$, and 0 for $\langle v_0, v_3 \rangle$, which maps to a different edge orbit. □

Tree decompositions. A tree decomposition organizes overlapping parts of a pattern $p = (V_p, E_p)$ as a tree $T = (V_T, E_T)$. Each tree node τ has a nonempty *bag* $B_\tau \subseteq V_p$, inducing the bag subpattern $p(\tau) = p[B_\tau]$. The bags satisfy: ① *node coverage*, $\bigcup_{\tau \in V_T} B_\tau = V_p$; ② *edge coverage*, both endpoints of every pattern edge occur in some bag; and ③ *connectedness*, the bags containing any pattern node form a connected subtree. The width of T is $w(T) = \max_{\tau \in V_T} |B_\tau| - 1$; the treewidth of p, denoted by $\mathsf{tw}(p)$, is the minimum width over all its tree decompositions.

Example 2.3: Figure 1(c) shows a tree decomposition of the running pattern with two bags. Together, the two bags cover all pattern nodes and edges. The nodes shared by the bags are $B_{\tau_0} \cap B_{\tau_1} = \{u_0, u_1, u_5\}$. The larger bag contains five nodes, so $w(T) = 4$. The matches of the two bag subpatterns can be computed separately and joined on the shared nodes $\{u_0, u_1, u_5\}$. □

Worst-case optimal join. WCOJ evaluates a multiway join one attribute at a time [24]. For graph matching, pattern nodes correspond to join attributes, and each pattern edge imposes an adjacency constraint on the corresponding data nodes. WCOJ thus extends a partial mapping one node at a time, following a backtracking procedure akin to Ullmann's algorithm [25], [26]. Given a node order $\pi = (\pi_1, \ldots, \pi_\ell)$, define the backward neighbors of π_i as $N^-_p(\pi_i, \pi) = \{\pi_j \in N_p(\pi_i) \mid j < i\}$. Given a partial mapping f of $\pi_1, \ldots, \pi_{i-1}$, the candidates of π_i are obtained by intersecting the adjacency constraints imposed by these already mapped neighbors:

$$\mathrm{Cand}_f(\pi_i) = \bigcap_{\pi_j \in N^-_p(\pi_i, \pi)} N_G(f(\pi_j)). \tag{3}$$

If $N^-_p(\pi_i, \pi) = \emptyset$, then $\mathrm{Cand}_f(\pi_i) = V$. For each candidate v, the evaluator checks the applicable mapping constraints, extends f with $\pi_i \mapsto v$, and recursively processes π_{i+1}. A complete mapping is emitted or aggregated after the last node is matched.

III. The pISO Counting Method

Under tISO counting [20], the ISO count is a weighted sum of tISO counts for the original pattern and correction patterns. tISO aggregates bag subpattern matches on shared nodes without enumerating complete matches. To keep this bag-wise processing efficient, tISO enforces injectivity only within individual bags. It does not check injectivity constraints between nodes u and u' for which no bag contains both. The resulting tISO count can therefore exceed the true ISO count and must be corrected. Recovering the ISO count requires the tISO method to consider every possible non-injective identification. For each such case, it constructs a smaller pattern and uses its ISO count to correct the non-injective tISO count. For example, Fig. 3 shows a pattern p with tree decomposition T having three bags; each τ_i contains u_i for $i \in \{0, 1, 2\}$ and the shared nodes u_3, u_4, u_5. Thus, each bag contains only one of u_0, u_1, u_2, and tISO does not constrain their matches across bags. Consequently, recovering the ISO count requires excluding four non-injective cases: one of the pairs $\{u_0, u_1\}$, $\{u_0, u_2\}$, and $\{u_1, u_2\}$ shares a data node, or all $\{u_0, u_1, u_2\}$ share the data node. Evaluating the corresponding correction counts introduces extra computation.

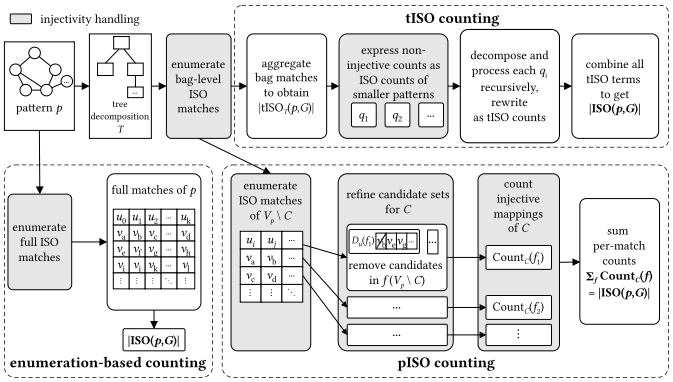


Fig. 2. Enumeration-based, tISO, and pISO counting.

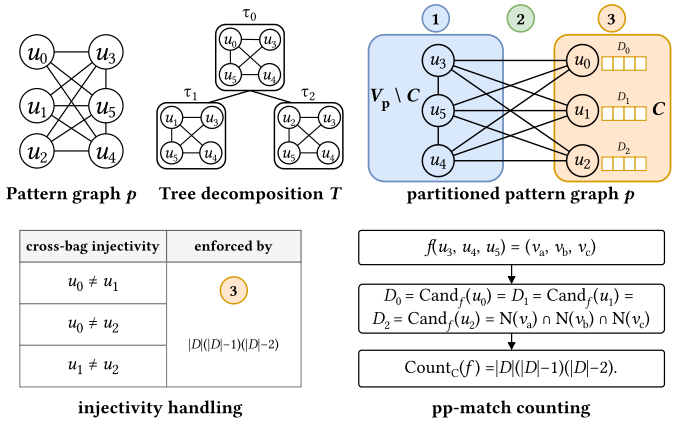


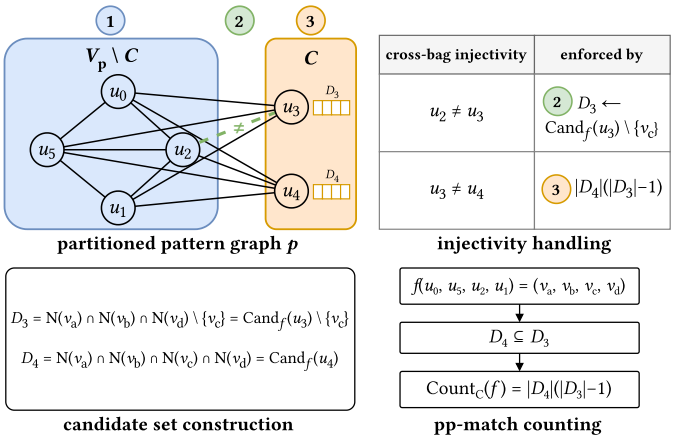
Fig. 3. An example of pISO counting.

A direct alternative is complete enumeration of the ISO matches of p. This approach obtains the ISO count without producing a non-injective intermediate count or evaluating correction patterns. However, complete enumeration can still be slower than tISO counting. Although tISO evaluates multiple patterns, it processes each one through a tree decomposition without enumerating complete matches.

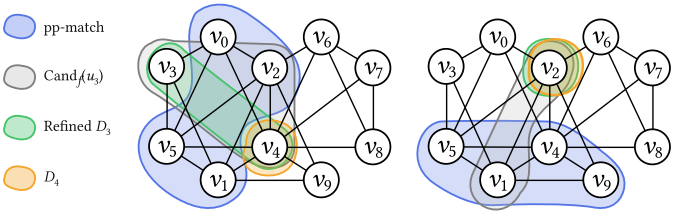
We propose pISO counting as a method that lies between complete enumeration and tISO counting. Fig. 2 compares the three workflows. Shaded modules handle injectivity constraints, whereas unshaded modules do not. Complete enumeration operates directly on p: it enumerates the complete ISO matches and thereby enforces all injectivity constraints. In contrast, both tISO counting and our pISO method decompose p and enumerate the ISO matches of individual bag subpatterns. tISO counting aggregates these bag matches into a tISO count and corrects its non-injective cases by recursively evaluating counts of smaller patterns. Our pISO method instead performs ISO matching over only part of the pattern. We will discuss how to determine which part of p is matched in Section III-A. For each partial match of p, we use an equation to count the mappings of the remaining pattern nodes that satisfy the injectivity constraints without enumerating them. We call such a partial p-match a *pp-match*, and call the number of mappings of the remaining pattern nodes its *pp-match count*. The ISO count equals the sum of the pp-match counts over all pp-matches. Thus, pISO computes the exact count from pp-matches and their injective extension counts, without enumerating complete matches or evaluating correction patterns.

To realize this workflow, we use the following observation: for a set C of pattern nodes with no edge between any pair, the number of its injective mappings can be computed directly from the candidate sets of its nodes without enumerating these mappings. We call C the *count set*, and the pp-match is for $V_p \setminus C$. We will give a construction of C in Section III-A. Based on this division, our pISO method proceeds in three steps. The three shaded modules along the pISO path in Fig. 2 correspond to these steps. First, we compute pp-matches over $V_p \setminus C$, which resolves the injectivity constraints for $V_p \setminus C$. Second, for each node in C, we exclude from its candidate set every data node already matched by the pp-match, thereby enforcing injectivity between C and $V_p \setminus C$. Finally, we use a single equation to count the mappings of C to enforce injectivity in C. In Fig. 3, the circled numbers 1–3 correspond to these three steps. Here, pISO method selects $C = \{u_0, u_1, u_2\}$ and constructs pp-matches over $V_p \setminus C = \{u_3, u_4, u_5\}$. Step 1 requires no additional injectivity check because u_3, u_4, u_5 occur together in every bag, whose ISO matches have already enforced their pairwise injectivity. Step 2 also requires no injectivity check because the bags have already enforced injectivity between $V_p \setminus C$ and C. The only injectivity constraints left unresolved after bag processing are $u_0 \neq u_1$, $u_0 \neq u_2$, and $u_1 \neq u_2$. Step 3 enforces all three with a single equation that counts the injective mappings of C without enumerating the candidate combinations for C or evaluating the tISO correction patterns. Fig. 4 applies the same procedure to the pattern p and tree decomposition T in Fig. 1. Here $C = \{u_3, u_4\}$. Step 1 constructs pp-matches over $V_p \setminus C = \{u_0, u_5, u_2, u_1\}$ and requires no injectivity check because their pairwise injectivity has already been checked when processing the bag τ_0. Step 2 enforces $u_2 \neq u_3$ by excluding the data node matched by u_2 from the candidates of u_3. Step 3 enforces $u_3 \neq u_4$ by directly counting mappings in which u_3 and u_4 select different data nodes.

The pp-match counting equation can take three forms. In the general form, the equation consists of multiple terms. Each term has a constant coefficient and a product of factors, where each factor is either the size of one candidate set or the size of an intersection of several candidate sets. If the candidate sets of all nodes in C satisfy certain structural properties, all candidate set intersection sizes can be replaced by individual candidate set sizes. If such properties hold for only part of C, the equation uses candidate set sizes for some terms and candidate set intersection sizes for the remaining terms. We begin with the base form and simplify this equation by comparing how the nodes in C connect to the nodes in $V_p \setminus C$. These connections determine the constraints used to construct each candidate set and hence whether two candidate sets are equal or one is contained in the other. If one candidate set is contained in



(a) The pISO counting framework. The count set $C = \{u_3, u_4\}$



(b) Two pp-matches and candidate sets for C

Fig. 4. pISO counting for the running example in Fig. 1.

another, their intersection has the size of the contained set, so the corresponding intersection factor can be replaced by an individual candidate set size. Applying these subset-based replacements produces the second form when all candidate set intersection sizes can be replaced and the third form when only some can be replaced.

A. PP-Matches and Candidate Sets for C

We next describe how pISO constructs C, computes pp-matches over $V_p \setminus C$, and derives the corresponding candidate sets from the preprocessed bags.

We first construct the count set C from a tree decomposition. The count set must be nonempty and contain no edge between any pair of its nodes. If a tree decomposition has only one bag, pISO falls back to enumeration-based counting. If a tree decomposition has a bag contained in another adjacent bag, we contract the edge between them and retain the containing bag. We therefore consider only tree decompositions with multiple bags in which no bag is contained in an adjacent bag.

For each bag τ in the tree decomposition T, if τ contains a node that occurs in no other bag, we select one such node and add it to C. For every selected node u, we denote its unique bag by $\tau(u)$.

Lemma 3.1: *The count set C constructed above satisfies the following properties: (i) $C \neq \emptyset$; (ii) no two nodes in C are adjacent in p; and (iii) for every $u \in C$, all its neighbors occur in its unique bag $\tau(u)$.*

Proof Sketch: Consider any degree-one bag τ with a neighbor τ'. Since B_τ is not contained in $B_{\tau'}$, $B_\tau \setminus B_{\tau'}$ is nonempty. For any $u \in B_\tau \setminus B_{\tau'}$, u occurs in no other bag. Otherwise, by the connectedness property, every bag on the path to another occurrence of u, including τ', would contain u, contradicting $u \notin B_{\tau'}$. Thus, every degree-one bag contributes a node to C. There are at least two degree-one bags, so C is nonempty. Any two distinct nodes in C come from different bags. If they were adjacent in p, the edge coverage property of a tree decomposition would require some bag to contain both nodes, contradicting that each selected node occurs only in its own bag. Finally, consider any $u \in C$ and one of its neighbors u'. By edge coverage, some bag contains both u and u'. Since u occurs only in $\tau(u)$, this bag must be $\tau(u)$. □

After fixing C, we compute pp-matches by reusing the materialized ISO matches of the bag subpattern represented by τ, as shown in the first two modules in the pISO counting panel in Fig. 2. Following the standard acyclic assembly along a tree decomposition [23], [27], [28], we project the materialized matches of each bag subpattern onto its nodes in $V_p \setminus C$ and join the projected matches on their shared pattern nodes to produce pp-matches. During this assembly, the join additionally enforces every previously unchecked injectivity constraint between nodes in $V_p \setminus C$. This avoids running the WCOJ on the data graph.

For each resulting pp-match f, we construct a candidate set for every $u \in C$. By construction and Lemma 3.1, u occurs in exactly one bag and all its neighbors also occur in this bag. A candidate of u must therefore be adjacent to $f(u')$ for every $u' \in N_p(u)$, as specified by Eq. (3). We obtain $\text{Cand}_f(u)$ from the materialized ISO matches g of $p(\tau(u))$. Specifically, among the matches satisfying $g(u') = f(u')$ for every $u' \neq u$ in $\tau(u)$, we collect the data nodes $g(u)$. This lookup reuses both the adjacency and injectivity checks already performed within the bag. The bag containing u may not contain every node covered by f, so its materialized matches may still yield $\text{Cand}_f(u)$ with data nodes already used outside that bag. We remove all data nodes used by f and define the resulting candidate set as

$$D_u(f) = \text{Cand}_f(u) \setminus f(V_p \setminus C). \tag{4}$$

This enforces injectivity between u and $V_p \setminus C$.

Example 3.1: Consider Fig. 4(a). The tree decomposition is in Fig. 1(c). Here $C = \{u_3, u_4\}$, and a pp-match is $f : (u_0, u_5, u_2, u_1) \mapsto (v_a, v_b, v_c, v_d)$. The bag τ_1 contains u_3 and does not contain u_2. The ISO matches in τ_1 check the adjacency constraints of u_3 and its injectivity against u_0, u_5, and u_1, but they do not exclude $f(u_2) = v_c$. We consequently remove v_c from $\text{Cand}_f(u_3)$ to obtain $D_3(f)$. In contrast, the bag τ_0 contains u_4 and every node in $V_p \setminus C$. Its materialized matches have already checked the injectivity of u_4 against $V_p \setminus C$, and hence $D_4(f) = \text{Cand}_f(u_4)$. Fig. 4(b) shows this procedure for two concrete pp-matches in the data graph. The blue regions identify the data nodes used by each pp-match. The green circles show the candidates retrieved for u_3, after which any candidate already used in the pp-match is removed to obtain $D_3(f)$. The orange circles show $D_4(f)$,

whose injectivity against the entire pp-match has already been enforced when processing bags. □

B. The counting equation

Let $C = \{u_1, \ldots, u_k\}$, and fix a pp-match f of $V_p \setminus C$. For each $u_i \in C$, let $D_i(f)$ be the candidate set constructed above. Since no two nodes in C are adjacent, choosing one data node from each of $D_1(f), \ldots, D_k(f)$ is subject only to pairwise injectivity. The individual candidate sets do not yet enforce this condition: two nodes $u_i, u_j \in C$ may still match the same data node in $D_i(f) \cap D_j(f)$. We characterize these non-injective mappings using partitions of C. In a partition $\mathcal{P}$, each block B represents nodes in C mapped to the same data node. The product over all blocks counts mappings satisfying the equalities represented by $\mathcal{P}$. Möbius inversion over the partition lattice cancels non-injective mappings and retains exactly the injective mappings:

$$\mathrm{Count}_C(f) = \sum_{\mathcal{P} \in \mathrm{Part}(C)} \mu(\mathcal{P}) \prod_{B \in \mathcal{P}} \left| \bigcap_{u_i \in B} D_i(f) \right|, \quad (5)$$

where $\mathrm{Part}(C)$ denotes the set of all partitions of C. The Möbius coefficient of $\mathcal{P}$ is $\mu(\mathcal{P}) = \prod_{B \in \mathcal{P}} (-1)^{|B|-1}(|B|-1)!$.

Example 3.2: For Fig. 3, consider the partial match of $V_p \setminus C, f : (u_3, u_4, u_5) \mapsto (v_a, v_b, v_c)$. No additional filtering of the local candidate sets in Eq. (3) is needed, so $D_i(f) = \mathrm{Cand}_f(u_i)$ for $i \in \{0, 1, 2\}$. For $C = \{u_0, u_1, u_2\}$, Eq. (5) expands to $\mathrm{Count}_C(f) = |D_0||D_1||D_2| + 2|D_0 \cap D_1 \cap D_2| - |D_0 \cap D_1||D_2| - |D_0 \cap D_2||D_1| - |D_1 \cap D_2||D_0|$.

The subset condition. We propose a *subset condition* under which the candidate set intersection sizes in Eq. (5) can be replaced by candidate set sizes. Consider two nodes $u_i, u_j \in C$. If $N_p(u_j) \subseteq N_p(u_i)$, the *subset condition* gives that $D_i(f) \subseteq D_j(f)$ and $D_i(f) \cap D_j(f) = D_i(f)$ for any f. By Eq. (4), every adjacency requirement used to construct $D_j(f)$ is also used to construct $D_i(f)$, and the same set $f(V_p \setminus C)$ is removed from both candidate sets, so $D_i(f) \subseteq D_j(f)$. The special case $N_p(u_i) = N_p(u_j)$ gives equivalent candidate sets and is recorded as two mutual subsets.

For each block B in a partition of C, we repeatedly remove a node $u_i \in B$ if another remaining node $u_j \in B$ satisfies $D_j(f) \subseteq D_i(f)$. This removal does not change the intersection represented by B, because $D_i(f) \cap D_j(f) = D_j(f)$. If the two candidate sets are equal, either node can be removed. Let B_{rem} denote the nodes remaining in B. Applying this deletion to every block in Eq. (5) gives

$$\mathrm{Count}_C(f) = \sum_{\mathcal{P} \in \mathrm{Part}(C)} \mu(\mathcal{P}) \prod_{B_{\mathrm{rem}} \in \mathcal{P}} \left| \bigcap_{u_i \in B_{\mathrm{rem}}} D_i(f) \right|. \quad (6)$$

Thus, an intersection size becomes an individual candidate set size when B_{rem} contains only one node; otherwise, only the intersection of the remaining candidate sets must be computed.

A subset condition chain eliminates all intersections. Without loss of generality, suppose $D_1(f) \subseteq D_2(f) \subseteq \cdots \subseteq D_k(f)$. In this case,

$$\mathrm{Count}_C(f) = \prod_{i=1}^{k} \big(|D_i(f)| - i + 1\big). \quad (7)$$

Lemma 3.2: *Let p be a pattern graph, G a data graph, and T a tree decomposition of p. Let $C = \{u_1, \ldots, u_k\}$ be the count set selected by a* pISO *plan over T, and let f be a pp-match of $V_p \setminus C$. Eq.* (5) *correctly computes* $\mathrm{Count}_C(f)$. *Applying the subset conditions in Eq.* (6) *preserves this count. If the candidate sets form a subset condition chain,* $\mathrm{Count}_C(f)$ *can be computed by Eq.* (7).

Proof Sketch: For a partition $\mathcal{P}$ of C, each block B requires all its nodes to select the same data node. The intersection factor associated with B counts the possible choices for that data node. Möbius inversion over the partition lattice cancels all non-injective mappings and retains exactly the injective mappings, proving Eq. (5). In Eq. (6), deleting u_i from a block when $D_j(f) \subseteq D_i(f)$ preserves its intersection because $D_i(f) \cap D_j(f) = D_j(f)$.

It remains to prove the chain form. Suppose that $D_1(f) \subseteq \cdots \subseteq D_k(f)$ and we select candidates one by one in this order and make all selected candidates distinct. After candidates have been selected for $u_1, \ldots, u_{i-1}$, all these $i-1$ distinct data nodes belong to $D_i(f)$ and are therefore the only values excluded by injectivity. Thus, every partial match has exactly $|D_i(f)| - (i-1)$ extensions to u_i. Multiplying these choices for $i = 1, \ldots, k$ gives Eq. (7). □

Example 3.3: In Fig. 3, each bag contains the same nodes u_3, u_4, u_5 in $V_p \setminus C$ and one node $u_i \in C$. Since the nodes in C have the same neighbors in $V_p \setminus C$ and all corresponding constraints are checked in bag processing, their candidate sets are identical: $D_0 = D_1 = D_2 = D$. These candidate sets form a subset condition chain, so Eq. (7) yields

$$\mathrm{Count}_C(f) = |D|(|D|-1)(|D|-2).$$

For Fig. 4, consider the pp-match and the candidate sets constructed above. Every node in D_3 is adjacent to v_a, v_b, and v_d and differs from v_c. Every node in D_4 satisfies these same adjacency constraints and is additionally adjacent to v_c. Because the data graph has no self-loops, adjacency to v_c also guarantees that the node differs from v_c. Therefore, every node in D_4 belongs to D_3, and $D_4 \subseteq D_3$. Applying Eq. (7) in the order D_4, D_3 gives

$$\mathrm{Count}_C(f) = |D_4|(|D_3| - 1).$$

Thus, both cases use only candidate set sizes. No set intersection is needed. □

C. Symmetry breaking

A symmetry-breaking rule consists of predicates of the form $u < u'$ and retains only mappings f satisfying $f(u) <$

$f(u')$ for every predicate, thereby pruning automorphically redundant ISO mappings. In pISO counting, predicates whose endpoints both lie in $V_p \setminus C$ prune pp-match generation, whereas predicates with exactly one endpoint in C filter that node's candidate set, reducing matching and counting costs.

Since pp-match counting counts mappings of C without enumerating them, it cannot explicitly check a predicate whose two endpoints are both in C. A symmetry-breaking rule may contain multiple predicates, and its reduction factor is valid only when the complete rule is enforced. We must therefore identify which complete rules can be enforced under the selected C and discard every rule that cannot be applied in its entirety. GraphPi [29] and GraphSet [30] overlook this requirement. They use Eq. (5) for node sets with no internal edges, fragment each symmetry-breaking rule into pairwise predicates $u_i < u_j$, and retain only those whose endpoints are both in $V_p \setminus C$. They then divide the aggregated count by a fixed number x derived from the retained predicates. This correction assumes that these predicates leave the same number of automorphic mappings in every case, which does not hold in general. For example, for a $K_{3,3}$ pattern, two isomorphic $K_{3,3}$ data graphs that differ only in node identifiers each contain one matched subgraph, but GraphPi and GraphSet return counts of one and two, respectively.

Selecting valid symmetry-breaking rules. We generate a complete set of symmetry-breaking rules for p through a stabilizer chain following [20], and retain only those usable under C. The automorphisms of p form a permutation group $\mathsf{Aut}(p)$ on V_p, i.e., a set of permutations with specific algebraic properties. For a permutation group Γ on V_p, the stabilizer of a node u is the subgroup $\Gamma_u = \{\gamma \in \Gamma \mid \gamma(u) = u\}$. A stabilizer chain is a sequence of such subgroups obtained by fixing one node at each step. Starting from $\Gamma_0 = \mathsf{Aut}(p)$, each step chooses an orbit ϑ_i of Γ_{i-1}, requires one node of ϑ_i to be smaller than the other $|\vartheta_i| - 1$ nodes, and fixes that node to obtain Γ_i. By the orbit–stabilizer theorem, $|\Gamma_{i-1}|/|\Gamma_i| = |\vartheta_i|$ [31]; the associated rule thus retains $1/|\vartheta_i|$ of the remaining ISO matches [20]. The rule associated with ϑ_i is usable if every predicate $u < u'$ in the rule satisfies $u \notin C$ or $u' \notin C$. We denote by ϑ_C the set of orbits from the stabilizer chain whose associated rules satisfy this condition.

Example 3.4: Consider again Fig. 3. The automorphisms of p can permute $\{u_0, u_1, u_2\}$, swap u_3 and u_4, and fix u_5. After C is fixed, the stabilizer chain construction starts from $\Gamma_0 = \mathsf{Aut}(p)$ and first uses the orbit $\vartheta_1 = \{u_0, u_1, u_2\}$, selects u_0 as its minimum, and generates $u_0 < u_1$ and $u_0 < u_2$. After fixing u_0, it uses $\vartheta_2 = \{u_1, u_2\}$ to generate $u_1 < u_2$. Lastly, after fixing u_0 and u_1, the remaining automorphisms can swap u_3 and u_4, so it generates $u_3 < u_4$. The rules associated with the first two chain orbits contain predicates whose two endpoints are both in C, so these complete rules are not applied. In contrast, $\vartheta_3 = \{u_3, u_4\}$ consists entirely of nodes in $V_p \setminus C$ and is usable. For the pp-match $f : (u_3, u_4, u_5) \mapsto (v_a, v_b, v_c)$, this rule enforces $v_a < v_b$. Thus, $\vartheta_C = \{\vartheta_3\}$ and $\prod_{\vartheta \in \vartheta_C} |\vartheta| = 2$.

Algorithm 1: pISO Counting

Input: pattern graph p, materialized bag subpattern matches, count set C, and chain orbits ϑ_C
Output: $|\mathsf{SubG}(p, G)|$: the count of matched subgraphs

$\mathsf{Sum}_C \leftarrow 0$;
foreach *pp-match f from joined bag matches* **do**
  foreach $u_i \in C$ **do**
    Obtain $\mathrm{Cand}_f(u_i)$ from matches of $p(\tau(u_i))$;
    $D_i(f) \leftarrow \mathrm{Cand}_f(u_i) \setminus f(V_p \setminus C)$;
    Apply symmetry rules associated with ϑ_C;
  if *candidate sets form a subset chain* **then**
    $c_f \leftarrow$ evaluate Eq. (7);
  else if *subset conditions hold in C* **then**
    $c_f \leftarrow$ evaluate Eq. (6);
  else $c_f \leftarrow$ evaluate Eq. (5) ;
  $\mathsf{Sum}_C \leftarrow \mathsf{Sum}_C + c_f$;
return $|\mathsf{SubG}(p, G)| = \frac{\prod_{\vartheta \in \vartheta_C} |\vartheta|}{|\mathsf{Aut}(p)|} \mathsf{Sum}_C$;

Since $|\mathsf{Aut}(p)| = 3 \cdot 2 \cdot 2 = 12$, Eq. (8) gives $|\mathsf{SubG}(p, G)| = \frac{2}{12} \sum_f \mathrm{Count}_C(f) = \frac{1}{6} \sum_f \mathrm{Count}_C(f)$. □

The symmetry predicates add constraints to the candidate sets of nodes in C and may refine the neighborhood-based equivalence and containment relations derived above. After selecting ϑ_C, we derive the final relations from all constraints used to construct each candidate set, including symmetry predicates. If every constraint for $D_j(f)$ also applies to $D_i(f)$, then $D_i(f) \subseteq D_j(f)$ for every pp-match f; if their constraint sets are identical, the two candidate sets are equivalent.

Lemma 3.3: *Let p be a pattern graph, G a data graph, and T a tree decomposition of p. Let C be the count set selected by a* pISO *plan over T, and let ϑ_C be the usable chain orbits defined above. The global subgraph count is*

$$|\mathsf{SubG}(p, G)| = \frac{\prod_{\vartheta \in \vartheta_C} |\vartheta|}{|\mathsf{Aut}(p)|} \sum_f \mathrm{Count}_C(f), \tag{8}$$

where f ranges over the pp-matches of $V_p \setminus C$.

Proof Sketch: By Lemma 3.2, $\mathrm{Count}_C(f)$ is the number of injective extensions of each pp-match f. The rules corresponding to ϑ_C are enforced in stabilizer chain order during pp-match generation or candidate set construction. Since each rule retains $1/|\vartheta|$ of the remaining mappings, $|\mathsf{ISO}(p, G)| = \prod_{\vartheta \in \vartheta_C} |\vartheta| \sum_f \mathrm{Count}_C(f)$. Finally, every subgraph occurrence corresponds to $|\mathsf{Aut}(p)|$ ISO mappings. Dividing the recovered ISO count by $|\mathsf{Aut}(p)|$ proves Eq. (8). □

Algorithm 1 gives the procedure for pISO counting using the materialized matches of the bag subpatterns. It initializes Sum_C and processes each pp-match f in turn. For every $u_i \in C$, it retrieves $\mathrm{Cand}_f(u_i)$ from the materialized matches of $p(\tau(u_i))$, removes the data nodes already used by f according to Eq. (4), and applies the symmetry predicates associated with the usable chain orbits to obtain $D_i(f)$. It evaluates the chain equation when all candidate sets form a subset condition

chain, the subset-simplified equation when only some subset conditions hold, and the base form otherwise. The resulting $c_f = \mathrm{Count}_C(f)$ is added to Sum_C. After all pp-matches have been processed, $\mathsf{Sum}_C = \sum_f \mathrm{Count}_C(f)$. Finally, the algorithm applies the symmetry and automorphism factors in Eq. (8) and returns the global subgraph count.

For node-orbit and edge-orbit counting, each output group is identified by the data node or edge matched to the selected orbit representative. The representative node, or both endpoints of the representative edge, must therefore be explicitly matched by the pp-match rather than placed in C. For node-orbit counting with representative o, pISO maintains one accumulator per data node and adds $\mathrm{Count}_C(f)$ to the accumulator indexed by $f(o)$ for each pp-match f. For edge-orbit counting with representative edge $\langle o, o' \rangle$, it similarly adds $\mathrm{Count}_C(f)$ to the accumulator indexed by $\langle f(o), f(o') \rangle$. We select only symmetry-breaking rules that preserve the aggregation group. To this end, we construct the stabilizer chain from automorphisms that fix o for node-orbit counting, or fix both o and o' individually for edge-orbit counting, and retain only complete rules usable under C. These rules prune symmetric mappings within each group without changing its aggregation key, and their reduction factors are compensated within each accumulator.

Further discussions of count-set selection and the relationship between symmetry breaking and orientation-based counting are provided in Appendix A.

IV. Time and Space Complexity of pISO

We analyze a fixed query and execution plan, treating pattern-dependent factors and arithmetic on counts as constant. As in Section II, $n = |V|$ and $m = |E|$. Let $w = w(T)$ be the width of the selected tree decomposition T of p, $s = |V_p \setminus C|$, and Δ the maximum data node degree. We assume sorted candidate lists; $\widetilde{O}$ suppresses logarithmic indexing and search factors. For a minimum-width decomposition, $w = \mathsf{tw}(p)$.

Time complexity. Bag subpattern evaluation and indexing take at most $\widetilde{O}(n^{w+1})$ time. Joining the projected matches explores at most n^{i-1} prefixes of length $i-1$, with at most $O(n)$ candidate list work per prefix. Summing over the s matched nodes gives $\widetilde{O}(n^s)$ time, including unsuccessful extensions, and at most n^s pp-matches.

For each pp-match, Algorithm 1 refines the candidate sets and evaluates the counting equation. Each node in C has a matched neighbor, so its candidate set has at most Δ nodes. For $m > 0$, refinement takes $\widetilde{O}(\Delta)$ time for a fixed pattern. The base form (Eq. (5)) computes each distinct intersection once and reuses it across terms, taking $\widetilde{O}(\Delta)$ time for a fixed pattern. The subset-simplified equation (Eq. (6)) reduces the required intersections but retains this worst-case bound. The subset chain equation (Eq. (7)) uses only stored cardinalities and takes $O(|C|)$ time, excluding candidate refinement. Thus, including graph access and output initialization, the coarse worst-case bound is $T_{\mathsf{pISO}} = \widetilde{O}\big(m + n^{w+1} + n^s\Delta\big)$. The $O(n)$ initialization cost is absorbed by n^{w+1} since $w \geq 0$. When counting uses only stored set sizes and needs no refinement scans, the last term reduces to n^s. Updating a global, node-orbit, or edge-orbit count takes constant time per pp-match.

Space complexity. For a fixed plan, processing pp-matches one at a time uses $O(n)$ auxiliary space for reusable candidate buffers and visitation marks. Materialized bag matches and their indexes require at most $O(n^{w+1})$ space. Including the input graph and global, VOC, or EOC output counters, the total space is $O(m + n^{w+1})$.

Comparison with SCOPE. Under the same bag-level evaluation model, let t be the number of original and correction terms used by SCOPE [20], and let w' be the maximum width across their selected decompositions. Summing the bag evaluation costs over these terms gives a coarse time bound of $\widetilde{O}(m + tn^{w'+1})$. Processing the terms takes $O(m + n^{w'+1})$ space, the same form as that of pISO but with the maximum decomposition width taken over all terms. Thus, pISO avoids correction term processing at the cost of pp-match generation and counting.

V. Optimizing Two Counting Methods

We propose PTCount, which selects a counting method for each ISO query using structural rules first and cardinality-based cost estimation only when these rules cannot decide. For tISO, these costs arise from bag evaluation across the original and correction patterns; for pISO, they arise from bag evaluation of the original pattern, pp-match generation, and equation-based counting.

A. Cost Model

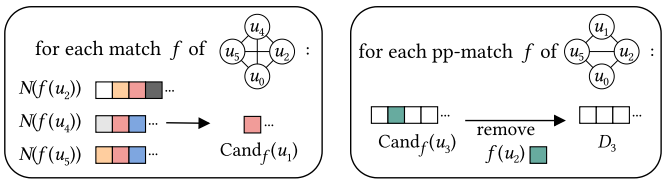
We model candidate list intersections for bag evaluation in both methods, and candidate construction and equation-based counting for each pp-match in pISO.

Intersection cost. Consider a plan that extends pattern nodes in an order $\pi = (u_1, \ldots, u_k)$. When the plan matches u_i, each partial mapping of $\pi_{<i} = (u_1, \ldots, u_{i-1})$ triggers one local extension step. This step constructs the candidates of u_i using the neighbor list intersection in Eq. (3). We write the intersection cost of this node extension step as

$$\mathsf{ICost}(u_i \mid \pi) = \mathsf{card}(p[\pi_{<i}]) \cdot \mathsf{L}(u_i \mid \pi_{<i}) / \mathsf{SF}(\pi_{<i}). \quad (9)$$

The term $\mathsf{card}(p[\pi_{<i}])$ is the estimated subgraph isomorphism cardinality of the first $i-1$ pattern nodes. We estimate such cardinalities using stratified graph sampling [32]. At position i, only the predicates whose endpoints both occur before u_i in π have been applied, and $\mathsf{SF}(\pi_{<i})$ estimates the pruning effect of this subset using an estimator independent of the data graph. Finally, $\mathsf{L}(u_i \mid \pi_{<i})$ estimates the lists participating in the local intersection. Following [23], we add the average data graph degree $\bar{d}$ for every previous neighbor u_j of u_i. The cost of an order sums Eq. (9) over its extension positions, and the cost of a tree decomposition plan further sums over its bags.

Example 5.1: In Fig. 5(a), bag τ_0 uses order $\pi = (u_0, u_5, u_2, u_4, u_1)$. Extending its four-clique prefix to u_1 intersects the neighbor lists of $f(u_5)$, $f(u_2)$, and $f(u_4)$, giving



(a) Intersection for u_1 (b) Candidate refinement for u_3

Fig. 5. An example of ICost and CCost on the running pattern in Fig. 1.

$\mathsf{L}(u_1 \mid \pi_{<5}) = 3\bar{d}$. The rule $u_2 < u_4$ has an estimated symmetry factor of two. Thus, $\mathsf{ICost}(u_1 \mid \pi) = \mathsf{card}(p[\pi_{<5}]) \cdot 3\bar{d}/2$. □

Cost of tISO counting. For an input pattern p and its selected tree decomposition plans, the cost of tISO counting is the total intersection cost of the original and correction terms:

$$\mathsf{Cost}_{\mathsf{tISO}}(p) = \sum_{q \in \mathcal{Q}_{\mathsf{tISO}}(p)} \mathsf{ICost}(q), \tag{10}$$

where $\mathcal{Q}_{\mathsf{tISO}}(p)$ contains the original tISO term and the recursively produced correction terms, and $\mathsf{ICost}(q)$ sums Eq. (9) over the bag extensions used to evaluate q.

Cost of pISO counting. The cost of a pISO plan with count set C consists of the bag-level intersection cost for p and the pp-match counting cost for C:

$$\mathsf{Cost}_{\mathsf{pISO}}(p, C) = \mathsf{ICost}(p) + \mathsf{CCost}(p, C). \tag{11}$$

Here $\mathsf{ICost}(p)$ sums Eq. (9) over the bag extensions of the selected pISO plan. The term CCost models the work performed for each pp-match. Let $C_{\mathsf{scan}} \subseteq C$ contain each node u_i whose candidate set must be scanned either to enforce an injectivity or symmetry predicate with a node in $V_p \setminus C$ that was not checked inside the bag containing u_i, or to obtain a cardinality required by the selected pp-match counting equation. Candidate set cardinalities already obtained during bag subpattern processing require no such scan. We estimate the counting cost as

$$\mathsf{CCost}(p, C) = \mathsf{card}(p[V_p \setminus C]) \cdot \mathsf{L}(C \mid V_p \setminus C), \tag{12}$$

where $\mathsf{card}(p[V_p \setminus C])$ is the estimated number of injective mappings reaching the counting stage. The per-mapping factor $\mathsf{L}(C \mid V_p \setminus C)$ is 1 when $C_{\mathsf{scan}} = \emptyset$; in this case, the selected equation incurs only constant work per mapping and CCost reduces to $\mathsf{card}(p[V_p \setminus C])$. Otherwise, it is the sum of $\mathsf{L}(u_i \mid V_p \setminus C)$ over $u_i \in C_{\mathsf{scan}}$. If u_i has only one edge to $V_p \setminus C$, we set $\mathsf{L}(u_i \mid V_p \setminus C) = \bar{d}$, as the work degenerates to scanning one neighbor list. In the general case, the required quantity is the filtered candidate set size after all constraints involving $V_p \setminus C$ are applied, rather than an input list size. We estimate it by $\mathsf{L}(u_i \mid V_p \setminus C) = \mathsf{card}(p[(V_p \setminus C) \cup \{u_i\}])/\mathsf{card}(p[V_p \setminus C])$.

Example 5.2: Fig. 5(b) shows the CCost of the same pISO plan. The count set is $C = \{u_3, u_4\}$, and each pp-match f is a match of $V_p \setminus C = \{u_0, u_1, u_2, u_5\}$. For each pp-match f, we scan $\mathrm{Cand}_f(u_3)$ once to remove $f(u_2)$ and obtain $D_3(f)$.

Algorithm 2: CompareTermCost

Input: tISO cost term 1 (p_1, L_1) and pISO cost term 2 (p_2, L_2)
Output: the term with the smaller cost, Equal, or Unknown

```
if |V_{p1}| < |V_{p2}| or (|V_{p1}| = |V_{p2}| and |E_{p1}| > |E_{p2}|) then return term 1 ;
else if |V_{p2}| < |V_{p1}| or (|V_{p1}| = |V_{p2}| and |E_{p2}| > |E_{p1}|) then return term 2 ;
else if p_1 ≇ p_2 then return Unknown;
else if L_2 ≠ d̄ then return term 2;
return the term with the smaller L or Equal if L_1 = L_2;
```

Thus, $C_{\mathsf{scan}} = \{u_3\}$, and $\mathsf{CCost} = \mathsf{card}(p[V_p \setminus C]) \cdot \mathsf{L}(u_3 \mid V_p \setminus C)$. □

B. Two-Step Optimizer

PTCount first performs a rule-based comparison using node and edge counts of subgraphs $p[\pi_{<i}]$, the values of L and SF in Eq. (9), and node and edge counts of the induced subgraph of $V_p \setminus C$ in Eq. (12).

For tISO counting, the selector collects the hardest ICost terms from the original and all correction patterns, using the last extension positions of their maximum-size bags. For pISO, the selector first compares its hardest ICost terms with tISO. If the tISO term is smaller, it selects tISO. If the pISO term is smaller, it additionally compares the pISO counting term over $V_p \setminus C$ with the competing tISO term. Algorithm 2 shows the comparison primitive CompareTermCost. A term consists of the cardinality of a subpattern of p and a non-cardinality factor. For an intersection term, the subpattern is $p[\pi_{<i}]$ and $L = \mathsf{L}(u_i \mid \pi_{<i})/\mathsf{SF}(\pi_{<i})$. For a pISO counting term, the subpattern is induced by $V_p \setminus C$ and $L = \mathsf{L}(C \mid V_p \setminus C)$. The algorithm first compares subgraph size and edge count. If the subgraph size and edge count are the same and the subgraphs are isomorphic, it compares the non-cardinality factors. The branch for $L_2 \neq \bar{d}$ favors pISO since in this case pISO scans a filtered candidate set result rather than the input lists of an ordinary intersection, and we treat this term as no larger than the corresponding list scanning work. If there are multiple hardest terms, we compare them pairwise and select a method only when every comparison favors the same method.

Example 5.3: Figure 5 illustrates a rule-based decision for the running ISO query. The hardest tISO intersection term in Fig. 5(a) extends u_1 after $\pi_{<5} = (u_0, u_5, u_2, u_4)$. The dominant pISO term in Fig. 5(b) is CCost over $V_p \setminus C = \{u_0, u_1, u_2, u_5\}$. Both subpatterns contain four nodes, but $p[\pi_{<5}]$ has six edges whereas $p[V_p \setminus C]$ has five. The denser subpattern is used as a proxy for a smaller cardinality, so the pISO counting term is judged larger and the selector chooses tISO. Figure 3 shows the opposite case. Here $C_{\mathsf{scan}} = \emptyset$ because the equivalent candidate sets require no refinement scan, and hence $\mathsf{L}(C \mid V_p \setminus C) = 1$ and the CCost is considered

to be small. The pISO plan has the same hardest bag term as the tISO plan. The selector therefore chooses pISO. □

If the rules cannot decide, PTCount estimates the unresolved cardinalities, evaluates the costs of both counting methods and selects the cheaper one.

VI. Related Work

Subgraph counting. A recent survey of algorithmic subgraph counting [7] classifies methods into exact and approximate approaches; exact methods are further grouped by their use of enumeration, counting equations, or pattern decomposition. Enumeration-based approaches count by explicitly visiting pattern occurrences [22], [33]–[41]. Equation-based and matrix-style approaches derive the target from related counts or orbit counting equations [8]–[13]. GraphPi [29] and GraphSet [30] use inclusion–exclusion for independent-node counting, while Fringe-SGC [42], a GPU-based system, counts fringe extensions through disjoint candidate-set regions. Our pISO instead combines subset-based equation simplification with complete, enforceable symmetry-breaking rules and tree-decomposition-based pp-match generation. A separate line decomposes the pattern and counts smaller subpatterns before assembling the final answer [14]–[20]. DISC converts ISO counts into linear combinations of homomorphism counts so that each term can use tree-decomposition-based aggregation [19]. SCOPE proposes tISO as an intermediate semantics that retains bag-local injectivity while supporting aggregation pushdown [20]. Approximate subgraph counting has also been studied through sampling [43]–[60], color coding [57], [58], machine learning [61]–[67], and algebraic methods [60]. BEACON benchmarks algorithmic and learning-based methods across diverse datasets and counting settings [68].

Subgraph matching. Subgraph matching enumerates ISO matches rather than only returning their counts. Existing single-machine systems are often organized around filtering, ordering, and enumeration, as summarized in recent studies and surveys [69]–[71]. Within this framework, prior work has studied candidate filtering, matching order selection, pruning, computation reuse, adaptive execution, decomposition-based enumeration, and redundancy-reduced enumeration [23], [25], [28], [29], [72]–[78]. There are also parallel and distributed subgraph matching and enumeration systems that decompose a query, compute partial matches, and assemble final matches under communication and memory constraints [79]–[82].

Tree decomposition and query processing. Tree decomposition and its variants control the complexity of constraint satisfaction and cyclic conjunctive queries [83]–[85]. Query processing combines these decompositions with aggregation, distributed execution, or caching [27], [86], [87]. DISC [19] and SCOPE [20] use decomposition for subgraph counting, whereas ASDMatch [28] and MDSE [23] use it for subgraph matching and enumeration.

VII. Experimental Evaluation

TABLE II
Data graph statistics.

Category	Dataset	Abbr.	$\|V\|$	$\|E\|$	d_G
Web	Web Spam [88]	ws	4,767	37,375	15.7
Web	Wiki-Vote [89]	wv	7,115	100,762	28.3
Collaboration	CA-AstroPh [90]	ca	18,771	198,050	21.1
Authorship	GitHub [88]	gh	121,706	439,849	7.2
Social	Gowalla [90]	gw	196,591	950,327	9.7
Collaboration	Mico [91]	mc	96,638	1,080,156	22.4
Social	Youtube [89]	yt	1,134,890	2,987,624	5.3
Web	Wiki-CH [88]	wc	1,930,270	8,956,902	9.3
Citation	US Patents [89]	up	3,774,768	16,518,947	8.8

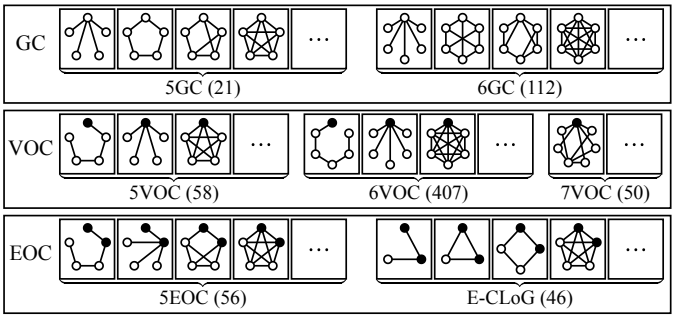


Fig. 6. Global and local counting query workloads.

A. Experimental Setup

Data graphs. We evaluate PTCount on nine real-world graphs, as summarized in Table II. The workload covers web graphs, social networks, collaboration/authorship graphs, and a citation graph. The last column reports the average degree $d_G = 2|E|/|V|$. The graphs range from small networks with a few thousand nodes to million-node graphs with more than sixteen million edges, and their average degrees range from 5.3 to 28.3. We use the preprocessed simple undirected versions of these graphs.

Query graphs. We evaluate global-count (GC), node-orbit-count (VOC), and edge-orbit-count (EOC) queries. The complete query families comprise 5GC (21 queries), 6GC (112), 5VOC (58), 6VOC (407), and 5EOC (56), where the leading number specifies the pattern size. For VOC and EOC, roots in different node or edge automorphism orbits of the same pattern define distinct queries. We also use the E-CLoG query set, comprising 46 edge-orbit queries selected from 3–5-node patterns. To evaluate larger patterns, we include 50 7-node node-orbit queries, denoted by 7VOC. Fig. 6 shows representative queries from each workload.

Baselines. We compare with three representative exact subgraph counting baselines. DISC is a general counting framework. It derives the ISO count from linear combinations of HOM counts [19]. SCOPE is the closest baseline: it introduces tISO equations and supports node-orbit counts [20]. E-CLoG is an equation-based method for edge-centric local graphlet counting and therefore serves as a baseline for edge-orbit counting workloads [13]. We do not separately include ESCAPE or EVOKE as a recent experimental study reports that SCOPE is faster than both, and SCOPE also supports

unrestricted pattern sizes [68]. Each baseline is evaluated on its supported workloads.

Settings. All experiments are conducted on a server running AlmaLinux 8.10. The machine has two Intel Xeon E5-2620 v4 processors, and 32 logical CPUs in total, together with 125 GiB of main memory. PTCount and the C++ baselines are compiled as C++17 programs using g++ 8.5.0 in Release mode with the -O3 optimization flag. The original DISC code is implemented in Scala. We implement a C++ version of DISC and verify that it is faster than the Scala implementation on our workloads, and therefore use the C++ version in the experiments. For a batch query workload, its running time is the sum of the running times of all constituent queries. Unless stated otherwise, average speedups are arithmetic means of per-instance speedups over instances completed by both methods; timeouts are reported separately.

B. Comparison with Baselines

Comparison with DISC. Fig. 7 compares PTCount with DISC on five ISO workloads: 5GC, E-CLoG queries, 5EOC, 6GC, and 7VOC. The figure includes 37 data graph–workload instances, of which DISC completes only 20 within the time limit. On these 20 common instances, PTCount is faster in every case, with an average speedup of $20.77\times$ and a range of $3.35\times$–$74.14\times$. For 6GC and 7VOC, ws is the only plotted data graph with a completed DISC run; PTCount achieves speedups of $74.14\times$ and $13.37\times$, respectively. DISC obtains each ISO answer through multiple HOM correction patterns. In contrast, PTCount selects tISO, which eliminates some corrections by enforcing injectivity within each bag, or pISO, which evaluates only the original pattern. This per-query choice avoids part or all of the additional pattern processing incurred by DISC.

Comparison with SCOPE. Fig. 7e compares the two methods on the 7VOC ISO workload. On ws, the two methods have comparable running times. On the other four larger data graphs, PTCount is consistently faster, achieving an average speedup of $1.26\times$. On YouTube, SCOPE reaches the 24-hour limit, while PTCount finishes in 80,261 seconds. PTCount supports global, node-orbit, and edge-orbit counting, whereas SCOPE supports only node-orbit counting.

Comparison with E-CLoG. Fig. 7b reports the workload supported by E-CLoG. On the five data graphs for which both systems produce finite results, PTCount is 4.64–$156.29\times$ faster, with a median speedup of $25.54\times$ and an average speedup of $49.35\times$. On Gowalla, E-CLoG times out while PTCount finishes in 1,207 seconds, yielding a speedup of at least $71.57\times$. Overall, PTCount completes all nine plotted data graphs, whereas E-CLoG has successful results on only five, showing that PTCount scales more consistently on this edge-centric query family.

Number of enumerated matches. The compared algorithms count by enumerating subpattern rather than all complete matches. We measure their work using the number of enumerated matches following [19], [20]. Fig. 8 reports the results. We omit instances on which the method exceeds the time limit. On the seven E-CLoG-query instances completed by both DISC and PTCount, PTCount enumerates between $2.01\times$ and $11.67\times$ fewer matches, with an average reduction of $5.24\times$. Against E-CLoG, it reduces the count by $32.60\times$–$252.68\times$ on five common instances. For 7VOC ISO, PTCount enumerates between $1.44\times$ and $1.53\times$ fewer matches than SCOPE on all five common instances; on ws, the only instance completed by DISC, the reduction over DISC is $20.37\times$.

TABLE III
ABLATION OF THE pISO OPTIMIZATIONS.

	Counting equation		Subset condition		Symmetry breaking	
Data graph	Speedup	OOT on/off	Speedup	OOT on/off	Speedup	OOT on/off
ws	$909.83\times$	0/23	$1.70\times$	0/0	$2.93\times$	0/0
wv	$562.87\times$	1/90	$2.24\times$	0/1	$3.54\times$	1/7
ca	$473.79\times$	0/87	$1.65\times$	0/0	$2.85\times$	0/1
gh	$133.59\times$	7/84	$2.14\times$	4/7	$3.24\times$	5/10
gw	$60.43\times$	26/137	$2.38\times$	12/19	$3.55\times$	22/34
yt	$37.18\times$	37/154	$2.85\times$	19/27	$3.86\times$	30/54
Overall	$477.11\times$	71/575	$2.12\times$	35/54	$3.29\times$	58/106

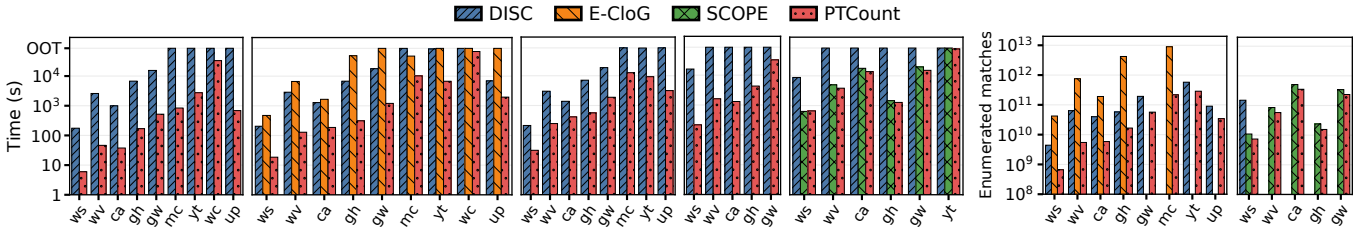
TABLE IV
TIME BREAKDOWN FOR FOUR PHASES IN pISO.

Equation form	$\lvert C\rvert$	processing bag	pp-match	refining candidates	counting C
Subset chain(Eq. (7))	2	16.68%	40.05%	26.96%	16.31%
	3	29.03%	37.25%	13.04%	20.69%
	4	28.17%	33.91%	8.74%	29.17%
Subset-simplified (Eq. (6))	3	2.95%	15.93%	35.07%	46.05%
	4	0.92%	12.57%	35.44%	51.07%
Base form (Eq. (5))	2	13.30%	18.01%	14.06%	54.62%
	3	20.60%	7.77%	4.43%	67.20%

C. Ablation Study

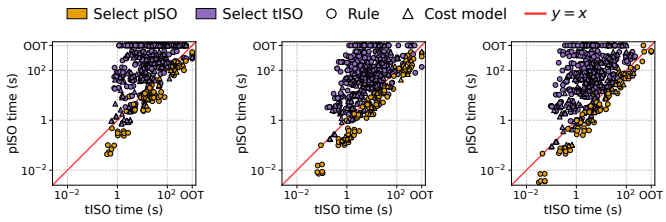
pISO optimizations. Table III isolates the three optimizations in the pp-match counting operator on the patterns in 5GC, 6GC, 5VOC, and 5EOC. For the counting equation ablation, the enabled configuration evaluates the counting equation over C; the disabled configuration counts only one node in C and accesses the remaining nodes through nested-loop traversal. For the subset condition ablation, we consider only queries that admit a subset condition. Enabling it uses Eq. (6) or Eq. (7), whereas disabling it forces the base form in Eq. (5). For symmetry breaking, we consider all queries for which pISO can apply at least one symmetry-breaking rule and compare executions with and without these rules. The timeout is 30 minutes, and each OOT on/off entry gives the number of timed-out runs with and without the corresponding optimization. The counting equation achieves a $477.11\times$ overall speedup and reduces OOTs from 575 to 71. The subset condition yields a $2.12\times$ speedup and reduces OOTs from 54 to 35. Symmetry breaking achieves a $3.29\times$ speedup and reduces OOTs from 106 to 58 by avoiding redundant enumeration of the same matched subgraph. Each optimization is effective on every data graph, showing consistent benefits across datasets with different structures and scales.

pISO execution time breakdown. Table IV breaks the execution into the four stages illustrated in Fig. 2: enumerating bag-level ISO matches, enumerating pp-matches over $V_p \setminus C$, refining the candidate sets of the nodes in C, and evaluating $\mathrm{Count}_C(f)$. We report 5GC, 6GC, 5VOC, and 5EOC query–



(a) 5GC (b) E-CLoG (c) 5EOC (d) 6GC (e) 7VOC

Fig. 7. Comparison with baselines using batch queries.



(a) gw (b) ca (c) wv

Fig. 9. Per-query performance and optimizer selections.

(a) E-CLoG (b) 7VOC

Fig. 8. Enumeration work.

data pairs on ws, wv, ca, gh, gw, and yt whose $|C| \in \{2, 3, 4\}$. For each pair, we normalize the four phase times by their sum and then average within each equation form and $|C|$. A smaller time fraction of the phase for counting C indicates lower equation evaluation overhead. The subset chain form, which eliminates all candidate set intersections, counts C with only 16.31%–29.17% of the total time, leaving bag processing and pp-match generation as the main costs. With only partial subset simplification, counting C remains the largest phase at 46.05%–51.07%; in the base form, its fraction further increases to 54.62%–67.20%. Thus, counting is most efficient under the subset chain condition, followed by a subset condition on only some pairs in C. Across all three equation forms, the relative time fraction of evaluating $\mathrm{Count}_C(f)$ increases with $|C|$.

Optimizer selection. Fig. 9 compares tISO and pISO on 5GC, 6GC, 5VOC, 6VOC, 5EOC, and 7VOC. The x-axis gives the forced tISO time and the y-axis gives the pISO time. Points above $y = x$ indicate queries for which tISO is faster, whereas points below $y = x$ indicate queries for which pISO is faster. Yellow and purple indicate that the optimizer selects pISO and tISO, respectively, while circles and triangles denote rule- and cost-based decisions, respectively. We cap timeouts at 10 minutes and omit pairs for which both methods time out. Among the 2,081 valid pairs, rules determine 1,396 selections, while estimated cardinalities determine the remaining 685. pISO is faster on 413 pairs (19.8%) and tISO on 1,668 (80.2%), and the optimizer selects the faster method for 1,843 pairs (88.6%). For context, always-tISO attains 80.2% accuracy, 8.4 percentage points below the optimizer. Of its 363 pISO selections, 269 are correct and 94 are incorrect. Over the 1,659 pairs completed by both methods, it achieves average speedups of 1.37× and 31.04× over always-tISO and always-pISO, respectively, while its runtime is 1.16× that of the oracle on average. For the 229 incorrect selections in this completed subset, the average slowdown is 2.15×. Rule-based and cost-based decisions achieve 91.0% and 83.5% accuracy, respectively. On the 363 pairs for which it selects pISO, pISO is 2.71× faster than tISO on average. On ws, gh, and yt, pISO and tISO win 387 and 1,646 of the 2,033 valid pairs, respectively, and the optimizer selects the faster method on 1,830 pairs (90.0%). The optimizer takes 0.37 s per query on average to select between pISO and tISO and build the plan, whereas PTCount takes 38.81 s for counting on average. The planning time is only 0.94% of counting time.

Structural characterization of pISO effectiveness. The pISO method is advantageous when equivalent or contained candidate sets allow the pp-match counting expression to be computed from candidate set sizes, eliminating set intersections within this operator. Among the 192 query–data graph pairs satisfying this condition, pISO is faster on 169 (88.0%). These patterns are degree-disassortative: their edges tend to connect nodes with dissimilar degrees, such as high-degree core nodes and low-degree peripheral nodes. We measure this tendency by the Pearson correlation between the degrees at the two endpoints of an edge. Following the common threshold [92], we regard $r \leq -0.5$ as strong degree disassortativity. Among the 108 corresponding pairs, pISO wins on 102 (94.4%). Conversely, patterns containing a long induced path favor tISO counting. The count set C can contain at most two nodes of this path. Consequently, $V_p \setminus C$ tends to retain a large, sparse path backbone, which can produce many pp-matches and amplify the refinement and counting work for each pp-match. We regard an induced 4-node path as a long path in this analysis. There are 9 out of 21 five-node pattern graphs and 79 out of 112 6-node pattern graphs that contain an induced 4-node path. Across the 204 query–data graph instances corresponding to these patterns where both methods finish within 10 minutes, tISO is faster on 192 (94.1%).

Appendix B covers six-node orbit counting, comparisons with GraphPi-, GraphSet-, and Fringe-style counters, memory usage, aggregation keys, and pattern structure.

VIII. Conclusion

We present PTCount, a tree-decomposition-based framework for exact global, node-orbit, and edge-orbit subgraph

counting. At its core is pISO, a partial-match-based method that computes only partial matches and derives the exact count using counting equations. PTCount selects between pISO and tISO for each query. Experiments on nine real-world data graphs show that PTCount achieves average speedups of $20.77\times$ over DISC and $49.35\times$ over E-CLoG. Compared to SCOPE's tISO method, the queries for which PTCount selects pISO achieve a $2.71\times$ average speedup. The optimizer selects the faster counting method with 88.6% accuracy, while method selection and query plan construction take only 0.37 seconds per query on average.

AI-Generated Content Acknowledgement

The authors used ChatGPT to assist with language polishing throughout the manuscript and code development. All AI-assisted text and code were reviewed and verified by the authors, who take full responsibility for the accuracy, originality, and integrity of the final paper.

References

[1] C. E. Tsourakakis, J. Pachocki, and M. Mitzenmacher, "Scalable motif-aware graph clustering," in *Proc. International Conference on World Wide Web (WWW)*. Perth, Australia: ACM, 2017, pp. 1451–1460.

[2] W. Hayes, K. Sun, and N. Przulj, "Graphlet-based measures are suitable for biological network comparison," *Bioinformatics*, vol. 29, no. 4, pp. 483–491, 2013.

[3] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626–688, 2015.

[4] R. Rotabi, K. Kamath, J. Kleinberg, and A. Sharma, "Detecting strong ties using network motifs," in *Proc. International Conference on World Wide Web Companion (WWW Companion)*. Perth, Australia: ACM, 2017, pp. 983–992.

[5] P. Barceló, F. Geerts, J. L. Reutter, and M. Ryschkov, "Graph neural networks with local graph parameters," in *Advances in Neural Information Processing Systems*, vol. 34. Virtual Event: Curran Associates, Inc., 2021, pp. 25 280–25 293.

[6] C. Qian, G. Rattan, F. Geerts, M. Niepert, and C. Morris, "Ordered subgraph aggregation networks," in *Advances in Neural Information Processing Systems*, vol. 35. New Orleans, LA, USA: Curran Associates, Inc., 2022, pp. 21 030–21 045.

[7] P. Ribeiro, P. Paredes, M. E. P. Silva, D. Aparício, and F. M. A. Silva, "A survey on subgraph counting: Concepts, algorithms, and applications to network motifs and graphlets," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 28:1–28:36, 2022.

[8] T. Hočevar and J. Demšar, "A combinatorial approach to graphlet counting," *Bioinformatics*, vol. 30, no. 4, pp. 559–565, 2014.

[9] ——, "Combinatorial algorithm for counting small induced graphs and orbits," *PLOS ONE*, vol. 12, no. 2, p. e0171428, 2017.

[10] I. Melckenbeeck, P. Audenaert, T. Michoel, D. Colle, and M. Pickavet, "An algorithm to automatically generate the combinatorial orbit counting equations," *PLOS ONE*, vol. 11, no. 1, p. e0147078, 2016.

[11] I. Melckenbeeck, P. Audenaert, D. Colle, and M. Pickavet, "Efficiently counting all orbits of graphlets of any order in a graph using autogenerated equations," *Bioinformatics*, vol. 34, no. 8, pp. 1372–1380, 2018.

[12] I. Melckenbeeck, P. Audenaert, T. V. Parys, Y. V. de Peer, D. Colle, and M. Pickavet, "Optimising orbit counting of arbitrary order by equation selection," *BMC Bioinformatics*, vol. 20, no. 1, pp. 27:1–27:13, 2019.

[13] V. S. Dave, N. K. Ahmed, and M. A. Hasan, "E-CLoG: Counting edge-centric local graphlets," in *Proc. IEEE International Conference on Big Data*. Boston, MA, USA: IEEE Computer Society, 2017, pp. 586–595.

[14] N. K. Ahmed, J. Neville, R. A. Rossi, and N. G. Duffield, "Efficient graphlet counting for large networks," in *Proc. IEEE International Conference on Data Mining (ICDM)*. Atlantic City, NJ, USA: IEEE Computer Society, 2015, pp. 1–10.

[15] D. Marcus and Y. Shavitt, "Efficient counting of network motifs," in *Proc. IEEE International Conference on Distributed Computing Systems Workshops (ICDCSW)*. Genova, Italy: IEEE Computer Society, 2010, pp. 92–98.

[16] L. A. A. Meira, V. R. Máximo, Á. L. Fazenda, and A. F. da Conceição, "acc-Motif: Accelerated network motif detection," *IEEE/ACM Trans. Comput. Biol. Bioinform.*, vol. 11, no. 5, pp. 853–862, 2014.

[17] A. Pinar, C. Seshadhri, and V. Vishal, "ESCAPE: efficiently counting all 5-vertex subgraphs," in *Proc. International Conference on World Wide Web (WWW)*. Perth, Australia: ACM, 2017, pp. 1431–1440.

[18] N. Pashanasangi and C. Seshadhri, "Efficiently counting vertex orbits of all 5-vertex subgraphs, by EVOKE," in *Proc. ACM International Conference on Web Search and Data Mining (WSDM)*. Houston, TX, USA: ACM, 2020, pp. 447–455.

[19] H. Zhang, J. X. Yu, Y. Zhang, K. Zhao, and H. Cheng, "Distributed subgraph counting: A general approach," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 2493–2507, 2020.

[20] Q. Li and J. X. Yu, "Fast local subgraph counting," *Proc. VLDB Endow.*, vol. 17, no. 8, pp. 1967–1980, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p1967-li.pdf

[21] "PTCount," https://github.com/magic62442/PTCount, accessed: November 6, 2026.

[22] J. A. Grochow and M. Kellis, "Network motif discovery using subgraph enumeration and symmetry-breaking," in *Research in Computational Molecular Biology*, ser. Lecture Notes in Computer Science, vol. 4453. Oakland, CA, USA: Springer, 2007, pp. 92–106.

[23] Q. Li, J. X. Yu, and Z. He, "Subgraph enumeration: Beyond tree decomposition," *Proc. VLDB Endow.*, vol. 19, no. 9, pp. 2303–2316, 2026.

[24] H. Q. Ngo, E. Porat, C. Ré, and A. Rudra, "Worst-case optimal join algorithms: [extended abstract]," in *Proc. ACM Symposium on Principles of Database Systems (PODS)*. Scottsdale, AZ, USA: ACM, 2012, pp. 37–48.

[25] S. Sun, X. Sun, Y. Che, Q. Luo, and B. He, "RapidMatch: A holistic approach to subgraph query processing," *Proc. VLDB Endow.*, vol. 14, no. 2, pp. 176–188, 2020.

[26] J. R. Ullmann, "An algorithm for subgraph isomorphism," *J. ACM*, vol. 23, no. 1, pp. 31–42, 1976.

[27] C. R. Aberger, S. Tu, K. Olukotun, and C. Ré, "EmptyHeaded: A relational engine for graph processing," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. San Francisco, CA, USA: ACM, 2016, pp. 431–446.

[28] Q. Li, J. Yu, and Z. He, "Subgraph matching: A new decomposition based approach," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4282–4294, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p4282-li.pdf

[29] T. Shi, M. Zhai, Y. Xu, and J. Zhai, "GraphPi: High performance graph pattern matching through effective redundancy elimination," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. Virtual Event: IEEE, 2020, pp. 1–14.

[30] T. Shi, J. Zhai, H. Wang, Q. Chen, M. Zhai, Z. Hao, H. Yang, and W. Chen, "GraphSet: High performance graph mining through equivalent set transformations," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2023, pp. 1–14.

[31] Á. Seress, *Permutation Group Algorithms*, ser. Cambridge Tracts in Mathematics. Cambridge University Press, 2003, vol. 152.

[32] W. Shin, S. Song, K. Park, and W. Han, "Cardinality estimation of subgraph matching: A filtering-sampling approach," *Proc. VLDB Endow.*, vol. 17, no. 7, pp. 1697–1709, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p1697-park.pdf

[33] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon, "Network motifs: Simple building blocks of complex networks," *Science*, vol. 298, no. 5594, pp. 824–827, 2002.

[34] S. Wernicke, "A faster algorithm for detecting network motifs," in *Algorithms in Bioinformatics*, ser. Lecture Notes in Computer Science, vol. 3692. Mallorca, Spain: Springer, 2005, pp. 165–177.

[35] S. Wernicke and F. Rasche, "FANMOD: a tool for fast network motif detection," *Bioinformatics*, vol. 22, no. 9, pp. 1152–1153, 2006.

[36] R. Itzhack, Y. Mogilevski, and Y. Louzoun, "An optimal algorithm for counting network motifs," *Physica A: Statistical Mechanics and its Applications*, vol. 381, pp. 482–490, 2007.

[37] Z. R. M. Kashani, H. Ahrabian, E. Elahi, A. Nowzari-Dalini, E. S. Ansari, S. Asadi, S. Mohammadi, F. Schreiber, and A. Masoudi-

Nejad, "Kavosh: a new algorithm for finding network motifs," *BMC Bioinformatics*, vol. 10, p. 318, 2009.

[38] P. M. P. Ribeiro and F. M. A. Silva, "g-tries: an efficient data structure for discovering network motifs," in *Proc. ACM Symposium on Applied Computing (SAC)*. Sierre, Switzerland: ACM, 2010, pp. 1559–1566.

[39] P. Paredes and P. M. P. Ribeiro, "Towards a faster network-centric subgraph census," in *Proc. IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. Niagara, ON, Canada: ACM, 2013, pp. 264–271.

[40] S. Khakabimamaghani, I. Sharafuddin, N. Dichter, I. Koch, and A. Masoudi-Nejad, "Quatexelero: an accelerated exact network motif detection algorithm," *PLOS ONE*, vol. 8, no. 7, p. e68073, 2013.

[41] M. Houbraken, S. Demeyer, T. Michoel, P. Audenaert, D. Colle, and M. Pickavet, "The index-based subgraph matching algorithm with general symmetries (ISMAGS): exploiting symmetry for faster subgraph enumeration," *PLOS ONE*, vol. 9, no. 5, p. e97896, 2014.

[42] C. L. Bradley, G. A. H. Alabandi, and M. Burtscher, "Fringe-SGC: Counting subgraphs with fringe vertices," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2025, pp. 1510–1523.

[43] N. Przulj, D. G. Corneil, and I. Jurisica, "Efficient estimation of graphlet frequency distributions in protein-protein interaction networks," *Bioinformatics*, vol. 22, no. 8, pp. 974–980, 2006.

[44] E. R. Elenberg, K. Shanmugam, M. Borokhovich, and A. G. Dimakis, "Beyond triangles: A distributed framework for estimating 3-profiles of large graphs," in *Proc. ACM International Conference on Knowledge Discovery and Data Mining (SIGKDD)*. Sydney, NSW, Australia: ACM, 2015, pp. 229–238.

[45] ——, "Distributed estimation of graph 4-profiles," in *Proc. International Conference on World Wide Web (WWW)*. Montréal, QC, Canada: International World Wide Web Conferences Steering Committee, 2016, pp. 483–493.

[46] P. Wang, Y. Qi, J. C. S. Lui, D. Towsley, J. Zhao, and J. Tao, "Inferring higher-order structure statistics of large networks from sampled edges," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 1, pp. 61–74, 2019.

[47] M. Rahman, M. A. Bhuiyan, and M. A. Hasan, "Graft: An efficient graphlet counting method for large graph analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 10, pp. 2466–2478, 2014.

[48] N. Kashtan, S. Itzkovitz, R. Milo, and U. Alon, "Efficient sampling algorithm for estimating subgraph concentrations and detecting network motifs," *Bioinformatics*, vol. 20, no. 11, pp. 1746–1758, 2004.

[49] M. Bhuiyan, M. Rahman, M. Rahman, and M. A. Hasan, "GUISE: uniform sampling of graphlets for large graph analysis," in *Proc. IEEE International Conference on Data Mining (ICDM)*. Brussels, Belgium: IEEE Computer Society, 2012, pp. 91–100.

[50] G. Han and H. Sethu, "Waddling random walk: Fast and accurate mining of motif statistics in large graphs," in *Proc. IEEE International Conference on Data Mining (ICDM)*. Barcelona, Spain: IEEE Computer Society, 2016, pp. 181–190.

[51] X. Chen, Y. Li, P. Wang, and J. C. S. Lui, "A general framework for estimating graphlet statistics via random walk," *Proc. VLDB Endow.*, vol. 10, no. 3, pp. 253–264, 2016.

[52] X. Chen and J. C. S. Lui, "Mining graphlet counts in online social networks," *ACM Trans. Knowl. Discov. Data*, vol. 12, no. 4, pp. 41:1–41:38, 2018.

[53] C. Yang, M. Lyu, Y. Li, Q. Zhao, and Y. Xu, "SSRW: A scalable algorithm for estimating graphlet statistics based on random walk," in *Database Systems for Advanced Applications (DASFAA), Part I*, ser. Lecture Notes in Computer Science, vol. 10827. Gold Coast, QLD, Australia: Springer, 2018, pp. 272–288.

[54] M. Jha, C. Seshadhri, and A. Pinar, "Path sampling: A fast and provable method for estimating 4-vertex subgraph counts," in *Proc. International Conference on World Wide Web (WWW)*. Florence, Italy: ACM, 2015, pp. 495–505.

[55] T. G. Kolda, A. Pinar, and C. Seshadhri, "Triadic measures on graphs: The power of wedge sampling," in *Proc. SIAM International Conference on Data Mining (SDM)*. Austin, TX, USA: SIAM, 2013, pp. 10–18.

[56] P. Wang, J. Zhao, X. Zhang, Z. Li, J. Cheng, J. C. S. Lui, D. Towsley, J. Tao, and X. Guan, "MOSS-5: A fast method of approximating counts of 5-node graphlets in large graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 1, pp. 73–86, 2018.

[57] M. Bressan, F. Chierichetti, R. Kumar, S. Leucci, and A. Panconesi, "Motif counting beyond five nodes," *ACM Trans. Knowl. Discov. Data*, vol. 12, no. 4, pp. 48:1–48:25, 2018.

[58] M. Bressan, S. Leucci, and A. Panconesi, "Motivo: Fast motif counting via succinct color coding and adaptive sampling," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1651–1663, 2019.

[59] A. Arpaci-Dusseau, Z. Zhou, and X. Chen, "Accurate and fast approximate graph pattern mining at scale," *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 93–107, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol18/p93-chen.pdf

[60] Q. Guo, J. Yang, W. Zhang, H. Wang, Y. Zhang, and X. Lin, "Algebraic subgraph counting," *Proc. ACM Manag. Data*, vol. 4, no. 4, pp. 1–27, 2026.

[61] K. Zhao, J. X. Yu, H. Zhang, Q. Li, and Y. Rong, "A learned sketch for subgraph counting," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Virtual Event, China: ACM, 2021, pp. 2142–2155.

[62] H. Wang, R. Hu, Y. Zhang, L. Qin, W. Wang, and W. Zhang, "Neural subgraph counting with Wasserstein estimator," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Philadelphia, PA, USA: ACM, 2022, pp. 160–175.

[63] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?" in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 10 383–10 395.

[64] X. Yu, Z. Liu, Y. Fang, and X. Zhang, "Learning to count isomorphisms with graph neural networks," in *Proc. AAAI Conference on Artificial Intelligence*, 2023, pp. 4845–4853.

[65] Y. Huang, X. Peng, J. Ma, and M. Zhang, "Boosting the cycle counting power of graph neural networks with I^2-GNNs," in *Proc. International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=kDSmxOspsXQ

[66] T. Fu, C. Wei, Y. Wang, and R. Ying, "DeSCo: Towards generalizable and scalable deep subgraph counting," in *Proc. ACM International Conference on Web Search and Data Mining (WSDM)*, 2024, pp. 218–227.

[67] Q. Guo, J. Yang, W. Zhang, H. Wang, Y. Zhang, and X. Lin, "Efficient and accurate subgraph counting: A bottom-up flow-learning based approach," *Proc. VLDB Endow.*, vol. 18, no. 8, pp. 2695–2708, 2025.

[68] X. Zhu, M. M. Najafi, C. Kosyfaki, X. Li, R. Cheng, and L. V. S. Lakshmanan, "[experiment, analysis, and benchmark] BEACON: A benchmark for efficient and accurate counting of subgraphs," in *Proc. IEEE International Conference on Data Engineering (ICDE)*, 2026, pp. 2683–2696.

[69] S. Sun and Q. Luo, "In-memory subgraph matching: An in-depth study," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Portland, OR, USA: ACM, 2020, pp. 1083–1098.

[70] Z. Zhang, Y. Lu, W. Zheng, and X. Lin, "A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 60:1–60:29, 2024.

[71] H. Jiang, S. Pandey, and H. Liu, "A comprehensive survey of subgraph matching: [experiments & analysis]," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–30, 2025.

[72] F. Bi, L. Chang, X. Lin, L. Qin, and W. Zhang, "Efficient subgraph matching by postponing Cartesian products," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. San Francisco, CA, USA: ACM, 2016, pp. 1199–1214.

[73] M. Han, H. Kim, G. Gu, K. Park, and W.-S. Han, "Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Amsterdam, The Netherlands: ACM, 2019, pp. 1429–1446.

[74] A. Mhedhbi and S. Salihoglu, "Optimizing subgraph queries by combining binary and worst-case optimal joins," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1692–1704, 2019.

[75] H. Kim, Y. Choi, K. Park, X. Lin, S.-H. Hong, and W.-S. Han, "Versatile equivalences: Speeding up subgraph query processing and subgraph matching," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Virtual Event, China: ACM, 2021, pp. 925–937.

[76] S. Sun, Y. Che, L. Wang, and Q. Luo, "Efficient parallel subgraph enumeration on a single machine," in *Proc. IEEE International Conference on Data Engineering (ICDE)*. Macao, China: IEEE, 2019, pp. 232–243.

[77] T. Jin, B. Li, Y. Li, Q. Zhou, Q. Ma, Y. Zhao, H. Chen, and J. Cheng, "Circinus: Fast redundancy-reduced subgraph matching," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 12:1–12:26, 2023.

[78] Y. Lu, Z. Zhang, and W. Zheng, "BSX: Subgraph matching with batch backtracking search," *Proc. ACM Manag. Data*, vol. 3, no. 1, pp. 15:1–15:27, 2025.

[79] L. Lai, L. Qin, X. Lin, and L. Chang, "Scalable subgraph enumeration in MapReduce: A cost-oriented approach," *VLDB J.*, vol. 26, no. 3, pp. 421–446, 2017.

[80] M. Qiao, H. Zhang, and H. Cheng, "Subgraph matching: On compression and computation," *Proc. VLDB Endow.*, vol. 11, no. 2, pp. 176–188, 2017.

[81] X. Ren, J. Wang, W.-S. Han, and J. X. Yu, "Fast and robust distributed subgraph enumeration," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1344–1356, 2019.

[82] Z. Yang, L. Lai, X. Lin, K. Hao, and W. Zhang, "HUGE: An efficient and scalable subgraph enumeration system," in *Proc. ACM International Conference on Management of Data (SIGMOD)*. Virtual Event, China: ACM, 2021, pp. 2049–2062.

[83] M. Grohe and D. Marx, "Constraint solving via fractional edge covers," *ACM Trans. Algorithms*, vol. 11, no. 1, pp. 4:1–4:20, 2014.

[84] G. Gottlob, G. Greco, N. Leone, and F. Scarcello, "Hypertree decompositions: Questions and answers," in *Proc. ACM Symposium on Principles of Database Systems (PODS)*. San Francisco, CA, USA: ACM, 2016, pp. 57–74.

[85] G. Gottlob, C. Okulmus, and R. Pichler, "Fast and parallel decomposition of constraint satisfaction problems," in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*. Yokohama, Japan: ijcai.org, 2020, pp. 1155–1162.

[86] H. Zhang, M. Qiao, J. X. Yu, and H. Cheng, "Fast distributed complex join processing," in *Proc. IEEE International Conference on Data Engineering (ICDE)*. Chania, Greece: IEEE, 2021, pp. 2087–2092.

[87] O. Kalinsky, Y. Etsion, and B. Kimelfeld, "Flexible caching in trie joins," in *Proc. International Conference on Extending Database Technology (EDBT)*. Venice, Italy: OpenProceedings.org, 2017, pp. 282–293.

[88] R. A. Rossi and N. K. Ahmed, "Network Repository," https://networkrepository.com, 2015, accessed: 2026-07-07.

[89] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," in *Proc. IEEE International Conference on Data Mining (ICDM)*. IEEE Computer Society, 2012, pp. 745–754.

[90] Stanford Network Analysis Project, "SNAP: Stanford Large Network Dataset Collection," https://snap.stanford.edu/data/index.html, 2026, accessed: 2026-07-07.

[91] M. Elseidy, E. Abdelhamid, S. Skiadopoulos, and P. Kalnis, "GRAMI: Frequent subgraph and pattern mining in a single large graph," *Proc. VLDB Endow.*, vol. 7, no. 7, pp. 517–528, 2014.

[92] M. Chiosa, T. B. Preußer, M. Blott, and G. Alonso, "AMNES: Accelerating the computation of data correlation using FPGAs," *Proc. VLDB Endow.*, vol. 16, no. 13, pp. 4174–4187, 2023.

[93] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized Algorithms*. Cham: Springer, 2015. [Online]. Available: https://doi.org/10.1007/978-3-319-21275-3

[94] M. Bressan, "Faster subgraph counting in sparse graphs," in *Proc. International Symposium on Parameterized and Exact Computation (IPEC)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 148. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:15.

[95] S. K. Bera, N. Pashanasangi, and C. Seshadhri, "Linear time subgraph counting, graph degeneracy, and the chasm at size six," in *Proc. Innovations in Theoretical Computer Science Conference (ITCS)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 151. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 38:1–38:20.

[96] D. Paul-Pena and C. Seshadhri, "Subgraph counting in subquadratic time for bounded degeneracy graphs," in *Proc. International Colloquium on Automata, Languages, and Programming (ICALP)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 334. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 124:1–124:18.

APPENDIX

A. Discussions

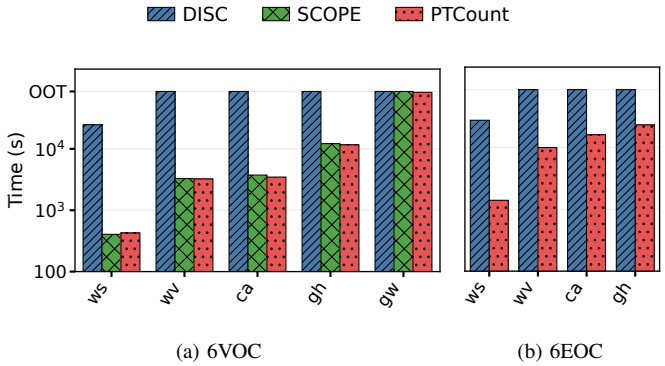
Choosing a maximum independent set as the count set. Choosing a maximum independent set as C minimizes the number of explicitly matched pattern nodes. Since the complement of an independent set is a vertex cover, $\mathsf{tw}(p) \leq |V_p| - \alpha(p) \leq |V_p \setminus C| = s$ [93], where $\alpha(p)$ is the maximum independent set size. Minimizing s reduces the exponent in the $n^s\Delta$ term of Section IV, but the overall cost also depends on the decomposition and counting equation.

First, this choice may conflict with the selected tree decomposition. Joining bag matches requires each shared pattern node to map to the same data node across the participating bags. Placing such a node in C removes it from the pp-matches and leaves the join without the corresponding data node to check this consistency. We therefore keep shared nodes outside C to retain the benefits of assembling pp-matches from materialized bag matches (Section III-A).

Second, a maximum independent set need not yield a simpler counting equation. The equation depends on containment relations among candidate sets, not just the number of nodes in C. A smaller count set whose candidate sets form a subset chain admits an equation using only their cardinalities, whereas a larger set without such containment relations requires candidate-set intersections. Count-set selection should therefore consider both compatibility with decomposition-based joins and the cost of evaluating the resulting equation.

Symmetry breaking and orientation-based counting. Orientation-based counting directs the data graph acyclically and combines counts for the corresponding acyclic orientations of the pattern. For a graph of degeneracy d, a degeneracy ordering gives an orientation with maximum outdegree at most d, bounding the branching of directed extensions. This supports DAG-tree-decomposition algorithms [94], linear-time counting of all patterns with at most five nodes on bounded-degeneracy graphs [95], and subquadratic-time counting of all patterns with at most nine nodes under the same assumption [96]. These bounds exploit sparsity as well as the structure of the directed patterns. A pattern can have multiple acyclic orientations, whose contributions must all be accounted for. Processing these directed cases introduces pattern-dependent work, although computations may be shared rather than executed independently. In practice, ESCAPE and EVOKE use degree-based orientations in specialized routines for selected basic structures and derive other counts through counting equations [17], [18]. They do not independently enumerate all orientations of every target pattern.

Symmetry breaking instead uses pattern automorphisms to prune redundant mappings. Under the same data-node order, a predicate $u_i < u_j$ on adjacent nodes can be enforced by the corresponding edge direction. However, symmetry predicates can also compare nonadjacent nodes, and a directed pattern can retain automorphisms. Thus, orientation and symmetry breaking overlap in their use of order constraints but serve different purposes. The former exploits bounded outdegree, whereas the latter removes automorphically equivalent mappings. In pISO, complete usable rules prune pp-match generation and filter candidate sets without splitting the pattern into directed cases. Their reduction factors are accounted for in Eq. (8). These rules do not by themselves provide the bounded-outdegree guarantees of degeneracy orientations.



(a) 6VOC (b) 6EOC

Fig. 10. Six-node orbit-counting batches.

B. Experiments

Six-node orbit counting. Fig. 10 compares batch running times for all 407 6VOC queries and 477 6EOC queries on ws, wv, ca, and gh, with gw additionally included for 6VOC. On ws, PTCount is $57.97\times$ and $19.71\times$ faster than DISC for 6VOC and 6EOC, respectively. On the other three graphs, PTCount completes both workloads within 24 hours, while DISC reaches the time limit. On the 6VOC comparisons, PTCount and SCOPE have comparable running times. The SCOPE-to-PTCount time ratio ranges from 0.95 to 1.08. PTCount is faster on four of five graphs. At the query level, Fig. 9 shows that pISO outperforms tISO on some queries.

Comparison with GraphPi, GraphSet, and Fringe-SGC. We compare the independent-node counting strategies of GraphPi [29], GraphSet [30], and Fringe-SGC [42] rather than their complete systems. GraphPi and GraphSet can return incorrect counts due to incomplete symmetry breaking (Section III), whereas Fringe-SGC is a GPU system. We implement their counting strategies in the same CPU executor, keeping pp-match generation, the count set C, symmetry rules, and normalization fixed. For each pp-match f, we compute its injective extension count using either our equations (Eqs. (5), (6), and (7)) or one of the three alternative counters. The GraphPi-style and GraphSet-style counters evaluate the same unsimplified inclusion–exclusion equation, using scalar two-pointer merging and SIMD comparisons, respectively, to materialize intersections. The Fringe-style counter scans the candidate sets to partition their union into disjoint membership regions, then counts injective assignments by combining allocations to these regions.

Table V compares these variants on 5GC, 6GC, 5VOC, and 5EOC, restricted to query–data pairs that our method completes within 600 seconds. Each alternative has the same time limit. Speedup is the alternative's execution time divided

TABLE V
SPEEDUPS OF pISO OVER ALTERNATIVE EXTENSION COUNTERS.

Counter	ws	wv	ca	gh	gw	yt	Overall
GraphPi-style	1.64×	1.92×	1.76×	1.87×	1.99×	2.31×	1.87×
GraphSet-style	1.65×	1.91×	1.78×	1.89×	2.02×	2.37×	1.89×
Fringe-style	2.68×	3.34×	2.86×	3.06×	3.45×	4.39×	3.17×

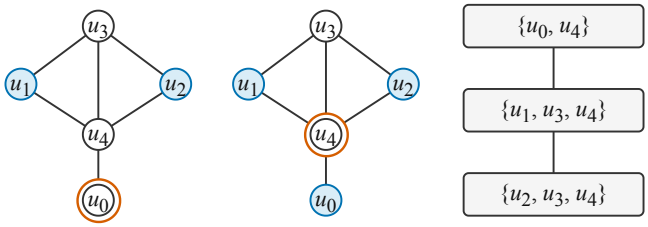
TABLE VI
AVERAGE MEMORY USAGE (MB).

Data graph	ca	gh	gw	mc	yt	wc	up
Input/output	33.48	37.96	93.66	222.14	252.36	793.92	1155.88
tISO	20.84	42.85	78.12	84.84	255.17	695.68	1314.66
PTCount	23.07	43.11	84.80	122.59	274.89	869.90	1306.02

by ours, averaged arithmetically over pairs completed by both methods. Our counter achieves non-trivial speedups over all three alternatives on every data graph with completed comparisons. Its key advantage is the subset condition: containment relations identified during planning reduce intersections to candidate-set sizes. A subset chain eliminates all intersections (Eq. (7)); partial containment reduces the remaining intersections (Eq. (6)). This avoids work that the alternatives perform through intersection materialization or disjoint-region construction.

Memory usage. Table VI reports average memory usage over the 5GC and 6GC patterns for which both methods finish within 600 seconds on each data graph. The input/output row reports the average peak memory of the input graph, data structures for accelerating intersections, and output arrays. During counting, tISO stores intermediate counts in arrays, whereas pISO stores materialized bag matches in tries. The tISO row reports its average peak memory usage for counting, after subtracting the corresponding no-count baseline. The PTCount row averages the same peak memory usage according to our optimizer's selection, using tISO for some patterns and pISO for others. All three rows use the same pattern set within each data graph. All values are in MB (10^6 bytes). Overall, PTCount increases counting memory usage by 13.78% relative to tISO on average across the seven data graphs, an acceptable memory overhead.

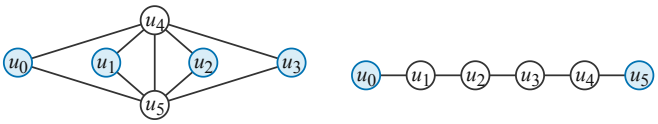
The effect of aggregation key. The aggregation key can reverse the relative performance of pISO and tISO even when the pattern is unchanged. Fig. 11 compares counting at the leaf u_0 and at the node u_4 of degree four. Double circles mark the aggregation key, and shaded nodes form C. Both plans use the decomposition in Fig. 11(c). On gw, pISO and tISO take 37.41 and 1.84 seconds for the leaf orbit, respectively, versus 0.72 and 1.49 seconds for the core orbit. The same reversal occurs on ca and wv. Both plans use counting equations based only on candidate set sizes. For the core orbit, $C = \{u_0, u_1, u_2\}$ leaves only the edge $\langle u_3, u_4\rangle$ to match. After removing the images already used by f, the candidate sets satisfy $D_1(f) = D_2(f) \subseteq D_0(f)$. Writing $a = |D_1(f)|$ and $b = |D_0(f)|$, the equation is $\mathrm{Count}_C(f) = a(a-1)(b-2)$. The first two factors assign distinct images to u_1, u_2, and the last factor excludes both images when counting choices for u_0. For



(a) Leaf orbit (b) Core orbit (c) Decomposition

Fig. 11. Node orbits and count sets.

the leaf orbit, u_0 must be explicitly matched as the aggregation key. Consequently, C shrinks to $\{u_1, u_2\}$, and the explicitly matched subpattern becomes the path through u_0, u_4, and u_3. Here $D_1(f) = D_2(f)$ also excludes the image of u_0. Recomputing a for this refined set gives $\mathrm{Count}_C(f) = a(a-1)$, which counts distinct images for u_1, u_2.



(a) Four triangles (b) P_6

Fig. 12. Contrasting pattern structures.

Structural interpretation. Fig. 12 illustrates the structural characterization in Section VII with two patterns of the same size. Shaded nodes form C. In Fig. 12(a), four triangles share the edge $\langle u_4, u_5\rangle$. Each triangle forms a bag with one private node, allowing $C = \{u_0, u_1, u_2, u_3\}$ while retaining the shared edge for explicit matching. Eight of the nine edges connect nodes of degrees two and five, giving strong degree disassortativity with $r = -0.8$. The four count nodes have the same neighborhood $\{u_4, u_5\}$ and hence identical candidate sets D for each mapping of the two core nodes. Writing $d = |D|$, their injective extensions are counted by $d(d-1)(d-2)(d-3)$. On gw, pISO takes 0.24 seconds versus 1.81 seconds for tISO. When low degree nodes connect to a small set of high degree nodes, some nodes in C may have identical or nested neighborhoods, linking degree disassortativity to subset conditions. Among 105 patterns in 5GC and 6GC with explicit plans, $|C| \geq 2$, and defined r, neighborhood containment within C occurs in 39 of 43 patterns with $r \leq -0.5$ (90.7%), versus 22 of 62 others (35.5%). In contrast, patterns containing a long induced path tend to favor tISO over pISO. For example, decomposing P_6 into edge bags leaves only its endpoints eligible for C, retaining a path of four nodes to match explicitly. Their different neighborhoods provide no containment relation between $D_0(f)$ and $D_5(f)$, so the base equation $|D_0(f)||D_5(f)| - |D_0(f) \cap D_5(f)|$ retains a candidate set intersection in addition to matching the path. On gw, tISO finishes in 435.36 seconds, whereas pISO reaches the 600 second timeout.